A Privacy-First Operating System with Integrated Knowledge Infrastructure

Design, Architecture, and Philosophy

Tim Jochen Kicker[a]

a University of Innsbruck, Austria

Abstract Linux is already a capable, secure, and privacy-respecting foundation for a desktop operating system. What it lacks is integration: applications store data in isolation, there is no cross-application query layer, and making a modern Linux desktop accessible and coherent for non-technical users remains an unsolved problem. This work presents the design and architecture of a Linux-based desktop operating system that addresses this gap without compromising the properties that make Linux worth building on.

At its core is a structured, local-first Knowledge Graph that records system activity as typed nodes and relationships, queryable across application boundaries and time. All system components (applications, background services, and an optional AI daemon etc.) access the graph through a single privileged daemon using cryptographically scoped capability tokens, with no component holding elevated trust by default.

The design is organized around five non-negotiables: local data by default, zero telemetry, full offline operation, least-privilege permissions throughout, and security that does not compromise usability. This work covers the full system stack from the Wayland compositor and application framework through the event infrastructure, permission model, profile isolation, and security architecture, including an explicit threat model and honest treatment of the limits of the proposed mitigations.

The system is currently in the design and planning phase. The contribution of this work is the architecture itself: a concrete, implementable blueprint for a privacy-preserving Linux desktop that treats the Knowledge Graph as infrastructure rather than a feature.

Keywords operating system, knowledge graph, privacy, user sovereignty, AI integration, Linux

Contents

1 Introduction **4**
1.1 Motivation . . . 4
1.2 Target Audience . . . 4
1.3 Design Principles . . . 5
1.4 Positioning . . . 5
1.5 Scope and Non-Goals . . . 5

2 System Overview **7**
2.1 Architecture Summarized . . . 7
2.2 Layer Responsibilities . . . 7
2.3 Why Rust . . . 9
2.4 Why Wayland . . . 9
2.5 Why Tauri . . . 10
2.6 Why Smithay . . . 11
2.7 Component Boundaries . . . 11
2.8 Communication Model . . . 12
2.8.1 Unix Sockets . . . 12
2.8.2 Protocol Buffers . . . 13
2.8.3 Knowledge Graph Rationale . . . 13
2.9 D-Bus Coexistence . . . 14

3 OS Foundation **15**
3.1 What is a Linux Distribution . . . 15
3.2 Base Distribution . . . 15
3.3 Update Strategy . . . 17
3.4 System Infrastructure . . . 19
3.5 Hardware Requirements . . . 20
3.6 Configuration Format . . . 21

4 Event System and Knowledge Graph **22**
4.1 Overview . . . 22
4.2 Kernel Monitoring with eBPF . . . 23
4.3 Event Bus . . . 24
4.4 Backpressure and Batching . . . 25
4.5 Knowledge Graph . . . 26
4.6 Retention Policy . . . 27
4.7 Temporal Filesystem View . . . 28

5 AI Layer **29**
5.1 Overview . . . 29
5.2 Cold-Start Behavior . . . 30
5.3 Provider Abstraction . . . 30
5.4 MCP Integration . . . 32
5.5 Interaction Model . . . 32

5.6 MCP Permission Model . . . 33
5.7 System Explanation Mode . . . 34

6 Desktop Environment and Design Language **35**
6.1 Architecture and IPC . . . 35
6.2 Compositor and Window Manager . . . 36
6.3 Shell and Application Framework . . . 37
6.4 Spotlight . . . 40
6.5 Notification System . . . 40
6.6 Screen Lock . . . 41
6.7 Multi-Monitor and HiDPI . . . 41
6.8 Design Language and Theming . . . 42

7 Apps, Modules, and the Store **43**
7.1 App Ecosystem . . . 43
7.2 Notification System . . . 45
7.3 Permission System . . . 46
7.4 Module System . . . 48
7.5 Store . . . 50

8 Platform, Profiles, and Security **52**
8.1 Account System . . . 52
8.2 Profile System . . . 54
8.3 Gaming and Windows Compatibility . . . 56
8.4 Security and Privacy . . . 57
8.4.1 Threat Model . . . 58
8.4.2 Full Disk Encryption . . . 59
8.4.3 Secure Boot, Unified Kernel Images, and Measured Boot . . . 59
8.4.4 App Sandboxing . . . 60
8.4.5 Knowledge Graph Security . . . 61
8.4.6 AI Security . . . 64
8.4.7 Audit Log . . . 66
8.4.8 Anomaly Detection . . . 67
8.4.9 Network Security . . . 67
8.4.10 Physical Security . . . 68
8.4.11 Memory Safety . . . 69
8.4.12 Multi-User Isolation . . . 69
8.5 Managed Environments . . . 70

9 Companion App and Health **71**
9.1 Companion App . . . 71
9.2 Health and Wellbeing . . . 73

10 Developer Experience, Sustainability, and Roadmap **75**
10.1 Developer Experience and Infrastructure . . . 75
10.2 Sustainability and Business Model . . . 79

10.3 Roadmap 81

1 Introduction

1.1 Motivation

Most Linux distributions are either conservative remixes of existing systems or highly opinionated tools for a specific niche. Neither describes this project.

The dominant desktop operating systems have a structural problem. Windows works against its own users: forced updates, advertisements in the start menu, and background telemetry that nobody enabled and cannot be fully disabled. The German Federal Office for Information Security (BSI) has studied this directly: as part of the SiSyPHuS research programme, the BSI found that required diagnostic data collection in Windows 10 and 11 runs continuously and cannot be turned off by the user [23]. macOS presents a different trade-off: its privacy posture is meaningfully better than Windows, but the ecosystem is deliberately constructed to keep users paying and dependent. A 2024 study published at CHI found that even opting out of Apple's analytics during device setup does not prevent data collection by individual system applications, and that the setup process itself involves at least eight distinct data-sharing decisions presented in ways that are not straightforward to review [47]. Every Linux distribution, however technically sound, has failed to reach people who simply want to use a computer without becoming a system administrator. The gap between technically capable and actually usable has not closed in thirty years.

This project exists to close that gap. It is a Linux-based operating system built from scratch with a different set of priorities: one that integrates a system-wide knowledge graph, AI interfaces, and modern tooling not as features bolted on top, but as foundational infrastructure. It takes a clear stance on user sovereignty, European software independence, and not being annoying to use. If an operating system is being built in 2025 anyway, there is no reason to repeat patterns from 1995.

1.2 Target Audience

The primary audience is people switching from macOS or Windows who want a genuinely good alternative: modern in appearance, functional out of the box, and respectful of their autonomy. Consumer-first, not enterprise-first. The macOS user tired of lock-in but unwilling to compromise on quality. The Windows user whose computer feels like it belongs to Microsoft more than to them. They do not want to configure. They want to arrive and work, and feel like the operating system is actually theirs.

Business deployment is a future consideration. The architecture supports managed environments and administrator policies. However, one line does not move: an administrator can configure defaults and restrict software; an administrator cannot surveil users or override their personal data ownership. Company needs will never come at the expense of user rights.

1.3 Design Principles

The following principles govern every design decision in this project. They are not aspirational language; they are constraints that break ties and override convenience.

The user's interests come first. No advertisements, no tracking, no dark patterns, no telemetry without explicit consent. This applies in managed environments too: an organization can configure and restrict, but cannot surveil. All system intelligence runs locally. The knowledge graph, AI context, and activity history do not leave the machine without an explicit user action. The system works fully offline, without registration, and without depending on services controlled by companies in other jurisdictions. Cloud features are opt-in extensions, not prerequisites. This is a technical commitment. It means the architecture enforces it, not just the privacy policy.

The system has to be genuinely good to use. Everything works on first boot. No hunting for drivers, no reading wiki pages to get audio working. The interface must look better than what users are coming from, because a poor interface is a barrier to entry regardless of what is underneath it. Small things matter more than large ones: a system that remembers what you were working on, or switches themes everywhere in one action, earns trust faster than an impressive feature nobody uses. Coherence matters too: shared configuration, shared theming, shared interfaces across all components. No part of the system should look like it was designed in a different decade by a different team.

New technology gets used when it makes sense, not because it is trending. AI and the knowledge graph are here because they genuinely change what a personal computer can do for the user. The goal is a useful, trustworthy system, not a clean architecture. Where established standards exist, this project builds compatibility bridges rather than replacements. Apps built for other Linux desktops run here; standard hardware interfaces work; nothing is deliberately broken for the sake of purity. The system can explain what it is doing, in plain language, in a place the user can see and act on it.

1.4 Positioning

Figure 1 maps the design space along two axes: how easy the system is to use for a non-technical user, and how much sovereignty the user has over their own data and software environment. These two properties have historically traded off against each other. Mainstream commercial systems prioritize usability but compromise on sovereignty, typically through telemetry, ecosystem lock-in, or both. The more privacy-conscious Linux distributions give users genuine control but impose a real usability cost. The upper-right quadrant, high usability *and* high user sovereignty, is where this project aims to land. No system targeting mainstream usability currently occupies it.

1.5 Scope and Non-Goals

The scope of this project is worth stating precisely, because several reasonable expectations turn out not to apply here.

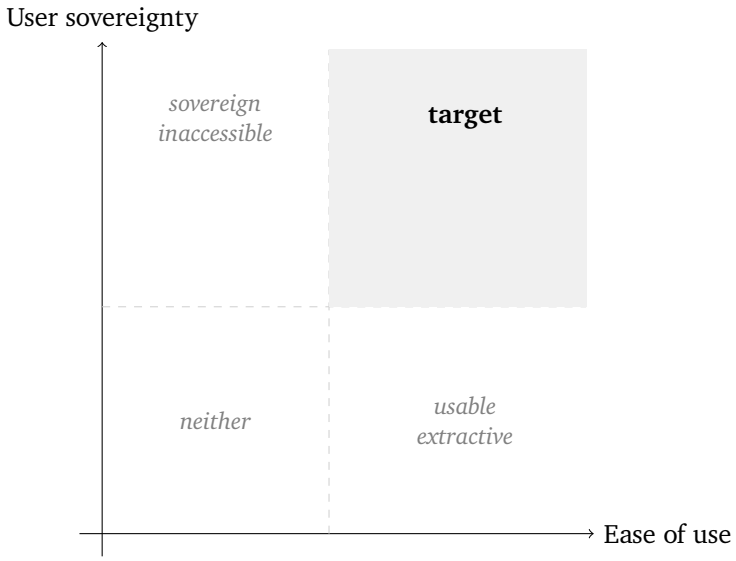

Figure 1 Design space: ease of use versus user sovereignty. The shaded quadrant is the project target. Mainstream commercial systems land in the lower-right (usable but extractive); privacy-oriented Linux distributions in the upper-left (sovereign but inaccessible). No system targeting mainstream usability currently occupies the upper-right.

Complexity is intentional. A system that works for non-technical users requires more infrastructure than a minimal one, not less. This is not a stripped-down environment optimized for simplicity of implementation.

Reproducibility is a goal; mandatory declarative configuration is not. Users should not need to manage configuration files to install software or change settings. Where declarative tooling is useful under the hood, it is used, but it is never a requirement imposed on the user.

The primary design constraint is the non-technical user switching from a commercial system. Advanced Linux users are well-served by the architecture and are welcome, but their preferences do not override decisions made for accessibility and usability. Those two audiences rarely conflict, but when they do, the newcomer wins.

Business deployment is a future consideration, not a present one. The architecture supports managed environments, but the system is not designed as an enterprise product. User rights are not negotiable regardless of deployment context.

The goal is a better alternative to existing systems, not a familiar imitation of them. Compatibility bridges are provided so that existing software works, but the interface, the architecture, and the underlying model are not constrained by what Windows or macOS do.

2 System Overview

2.1 Architecture Summarized

The system is organized as a layered stack. Lower layers provide services to higher layers; nothing reaches past its immediate neighbor. The bottom four layers are always present: the OS base, the desktop environment, the messaging and integration infrastructure, and the Knowledge Graph. Above those sit the system's consumers — the applications, daemons, and optional AI daemon that read from and write to the graph. Figure 2 shows the full structure.

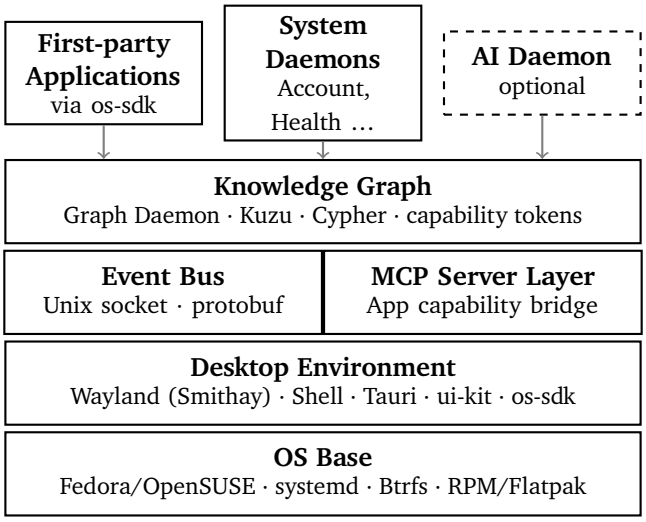

Figure 2 System layer structure. The bottom four layers are always present and initialize in order from bottom to top. The top row shows the three classes of graph consumer: first-party applications (connected via os-sdk), system daemons (Account, Health, Audit, and others), and the AI daemon (dashed, optional). All three connect to the Graph Daemon over a Unix socket with per-application capability tokens; none of them bypasses the daemon or accesses Kuzu directly. The AI daemon is not architecturally privileged over the others.

2.2 Layer Responsibilities

OS Base. The foundation is a standard Linux distribution providing the kernel, init system, package manager, and base userspace tools. Nothing at this layer is custom-built. The system builds on top of it; it does not replace it. The choice of base distribution, the update strategy, and the concrete packages involved are covered in detail in Section 3.

Desktop Environment. The Wayland compositor and the shell sit at this layer. This is what the user directly interacts with: windows, the taskbar, the Spotlight launcher, notifications, and the lock screen. All user-facing applications in this project are built with *Tauri* [77], a framework described in Section 2.5. The compositor itself is built on *Smithay* [72], described in Section 2.6.

Event Bus. The internal message bus handles all communication between custom system components. It receives, validates, and routes structured events from kernel monitoring, the display system, and first-party applications. It runs as a background service over *Unix sockets* (see Section 2.8.1) using *Protocol Buffers* for schema-enforced serialization (see Section 2.8.2). Its role is purely to move and validate data; it does not interpret or store anything.

MCP Server Layer. The *Model Context Protocol* [2] (MCP) is an open standard published by Anthropic in 2024 for exposing application capabilities as callable tools that AI models can invoke. In this system first-party applications expose MCP servers as part of their standard implementation via os-sdk, giving any MCP-compatible client a structured, permission-gated way to act on the system. The MCP layer is not specific to the AI daemon; any client that implements the protocol can use it.

Knowledge Graph. The graph database is the system's long-term memory: a structured record of everything that happens on the machine, queryable across time, application boundaries, and data types. The *Graph Daemon* is the sole reader and writer of the underlying database; no other process opens the database files directly. Every component that needs graph data connects to the daemon over a Unix socket and presents a cryptographically signed capability token that encodes its permitted query scope. Why a graph database rather than a relational one, and why *Kuzu* [48] specifically, is explained in Section 2.8.3.

First-party applications. Every first-party application written for this system connects to the Graph Daemon via the typed client bundled in *os-sdk*. Opening the File Manager, searching in Spotlight, or checking System Monitor all translate to parameterized Cypher queries over a Unix socket. The application receives structured results and renders them; it never touches the Kuzu files directly, and the capability token it presents limits which node types and relations it can read.

System daemons. Background daemons such as the Account Daemon, Health Daemon, and Audit Daemon are also graph clients. They write structured data into the graph (account events, health observations, audit records) and read from it when their function requires cross-domain context. Like applications, each daemon holds a capability token with the minimum scope it needs; graph access is not elevated for daemons by default.

AI daemon (optional). The AI daemon is a system daemon like the others. It connects to the Graph Daemon via the same Unix socket interface, presents the same style of capability token, and receives the same enforcement from the daemon. What distinguishes it is not a privileged protocol position but a broader read permission at higher user-configured access levels (see Section 8.4.6). It additionally orchestrates tool calls via the MCP layer and mediates between user requests and model providers, local or cloud, behind a common Rust trait interface (see Section 2.3). Disabling the

AI daemon removes AI features and nothing else; the graph, the applications, and the other daemons are unaffected.

2.3 Why Rust

All system daemons and Tauri application backends in this project are written in *Rust*, a systems programming language developed by Mozilla Research with its 1.0 release in 2015. Understanding why requires a brief look at what systems programming has historically looked like.

C and C++ have dominated operating system components, daemons, and performance-critical software for decades. Their advantage is direct control over memory and hardware with minimal overhead. Their liability is that this control is entirely manual: the programmer allocates memory and is responsible for freeing it correctly, without using it afterward, without reading past its bounds, and without sharing it unsafely across threads. Memory safety bugs (use-after-free, buffer overflows, data races) are the single largest source of exploitable vulnerabilities in system software. Google's Project Zero and Microsoft's Security Response Center have independently reported that roughly 70% of serious security vulnerabilities in their codebases are memory safety issues [55].

Go is a more recent alternative with garbage collection and a simpler concurrency model, but its garbage collector introduces unpredictable pause latency unsuitable for low-level system components, and its runtime overhead is significant compared to C or Rust.

Rust enforces memory safety through a compile-time ownership and borrowing system. An entire class of bugs that would be exploitable vulnerabilities in C simply cannot compile in Rust; the compiler rejects them. This is not a runtime check with performance overhead; it is a zero-cost compile-time guarantee. Rust has no garbage collector and achieves performance comparable to C and C++.

For this project the consequence is concrete: every daemon runs continuously, handles untrusted input from applications, and manages sensitive data including credentials and graph queries. Writing them in Rust eliminates an entire vulnerability class without sacrificing performance.

A *Rust trait* is the language's mechanism for defining a shared interface that multiple concrete types can implement, analogous to an interface in Java or a pure abstract class in C++. When this document refers to the AI provider abstraction as a Rust trait, it means that every AI backend implements the same set of methods, and the rest of the codebase calls only those methods without knowing which concrete backend is active at runtime.

2.4 Why Wayland

Wayland is a display server protocol: the specification that defines how applications communicate with the component responsible for drawing windows on the screen.

X11 (the X Window System) was designed in 1984 and has been the foundation of graphical Linux desktops ever since. Its architecture reflects its era: the display server

is a separate process, and by default any application can inspect the input events and window contents of any other application on the same display. One X11 application can read keystrokes typed into another application's window. This is a keylogger-class capability that cannot be removed without a protocol redesign, and it makes X11 structurally incompatible with the isolation guarantees this project requires.

Wayland redesigns this from scratch: each application communicates only with the compositor, not with other applications. Input events are delivered only to the window that currently has focus. An application cannot read another application's keystrokes or window contents. This isolation is the default behaviour, not an optional hardening measure.

Wayland also has a cleaner model for multi-monitor and HiDPI configurations: each output carries its own scale factor, and applications receive this information directly and render at the correct resolution automatically.

The compatibility cost is real. Wayland addresses it through *XWayland*, an X11 compatibility server that runs as a Wayland client. Legacy X11 applications work inside XWayland without modification. X11's security limitations apply within the XWayland boundary but are contained there and do not affect native Wayland applications.

2.5 Why Tauri

Tauri is a framework for building desktop applications with a Rust backend and a web-based user interface. A Tauri application consists of a Rust process that handles system interactions, file access, and IPC, and a *WebView* that renders the UI using standard web technologies: HTML, CSS, and TypeScript. The two halves communicate through a typed command bridge: the frontend calls Rust functions via `invoke()`, and Rust pushes events to the frontend via Tauri's event emitter. No ad-hoc serialization is needed; the bridge handles it.

The dominant alternative is *Electron*, which takes the same split but bundles an entire Chromium browser and a Node.js runtime with every application. A minimal Electron application ships roughly 150 MB of runtime dependencies and consumes around 300 MB of RAM at idle. On a system with several Electron applications running, the memory cost is substantial.

Tauri uses the operating system's existing WebView implementation instead of bundling one. On Linux this is *WebKitGTK*, the GTK port of the WebKit rendering engine. WebKitGTK is already present on any modern Linux desktop; a Tauri application does not add it. A Tauri application binary is typically 5–10 MB and uses a fraction of the RAM that an equivalent Electron application would require.

Building the UI in a native toolkit such as *GTK4* or *Qt* was considered and rejected for two reasons. First, maintaining visual consistency across components developed independently is significantly harder with a native toolkit than with a shared CSS design system where a token change propagates everywhere. Second, the developer pool for TypeScript and CSS is larger than for GTK or Qt development, which matters for an open-source project expecting community contributions. The trade-off is that WebView rendering is slightly less efficient than native rendering for highly dynamic

content, but this is not a relevant concern for the settings panels, file browsers, and launchers that make up this system's UI.

Tauri version 2.0, released in October 2024, is used throughout this project. Version 2.0 introduced a revised permission system for Tauri commands (replacing the allowlist model of 1.x with a finer-grained capability model), official mobile support, and improved IPC ergonomics. The capability model in 2.0 aligns well with this project's general approach to least-privilege permissions.

2.6 Why Smithay

Smithay is a Rust library for building Wayland compositors. It provides the low-level plumbing: Wayland protocol handling, DRM/KMS interfaces for direct display hardware access, input processing via *libinput*, and buffer management. Compositor logic such as window placement, focus rules, and workspace behaviour is built on top of Smithay rather than dictated by it.

The alternative is *wlroots*, a C library used by compositors including Sway and Hyprland. wlroots is mature and widely deployed, but integrating it into a Rust codebase requires C FFI (Foreign Function Interface) bindings, which partially undermines the memory safety argument for using Rust: the boundary between Rust and C code requires `unsafe` blocks that the Rust compiler cannot verify. Smithay is written in Rust and integrates cleanly with the rest of the codebase without unsafe boundaries.

The *COSMIC* desktop from System76 is a production Rust-first Linux desktop with comparable technical goals and uses Smithay as its compositor foundation. It serves as a useful reference for what a Smithay-based desktop looks like in practice.

2.7 Component Boundaries

The system's components divide into two categories: background services and user-facing applications.

Background services are called *daemons* in Unix terminology: processes with no visible window that are started at boot by *systemd* and automatically restarted if they crash. systemd is the standard init system and service manager on modern Linux distributions. It handles starting services in dependency order at boot, logging their output to the journal, and enforcing resource and sandboxing constraints via its service unit configuration. All daemons in this project are written in Rust and run under tight systemd sandboxing: each service unit declares exactly the filesystem paths, syscalls, and network access it requires, and everything else is denied at the kernel level.

User-facing applications are built with Tauri. Every first-party application is built on two shared libraries. *ui-kit* is a TypeScript component library based on *shadcn/ui* (a set of accessible, unstyled components built on Radix UI primitives) styled with the project's shared design token system using *Tailwind CSS*. Tailwind is a utility-first CSS framework where styles are applied via class names rather than separate stylesheets; combined with CSS custom properties for design tokens, a theme change propagates to all running applications without a reload. *os-sdk* is a Rust crate that provides ready-made clients for the Graph Daemon and Event Bus, MCP server boilerplate, and the

Table 1 Background services (daemons)

Service	Responsibility
Event Bus	Receives, validates, and routes internal events between components
Graph Daemon	Sole reader/writer of the Knowledge Graph database
Account Daemon	Credential management and login token handling
AI Daemon	Model providers, MCP orchestration, graph queries (optional)
Module Runtime	Manages the lifecycle of shell extensions
Transfer Daemon	Controls data movement between user profiles
Audit Daemon	Sole writer to the append-only audit log
Health Daemon	Aggregates and stores health and wellbeing data
Compositor	Draws windows and manages the display (Wayland compositor)

shared TOML configuration system. A new first-party application starts with these two libraries and has graph access, event emission, an MCP server, and a consistent UI without writing any infrastructure code.

Table 2 User-facing applications

Application	Notes
Shell	Taskbar, Spotlight launcher, notification center, lock screen
Settings	System-wide configuration: theming, permissions, AI level, accounts
File Manager	First-party file browser with graph-aware features
Store	App and module discovery, installation, and updates
Terminal	First-party terminal emulator with MCP integration
Wine Manager	Windows compatibility management UI

2.8 Communication Model

2.8.1 Unix Sockets

All inter-component communication uses *Unix domain sockets*, a communication mechanism built into the Linux kernel. A Unix socket behaves like a network socket in its API but exists only on the local machine: it is represented as a file path in the filesystem (for example `/run/graph-daemon/graph.sock`), and only processes on the same machine can connect to it. There is no network stack involved; data passes directly through the kernel with latency in the range of a few microseconds.

D-Bus, the standard Linux IPC mechanism discussed in Section 2.9, was not chosen for internal component communication: it introduces a central broker process as an intermediary, adds marshalling overhead, and its type system is not well suited to the structured binary messages used here. TCP loopback sockets were also considered and

rejected: they go through the full network stack, carry more overhead, and expose a port reachable by any process on the machine.

2.8.2 Protocol Buffers

Data transmitted over the internal sockets is serialized with *Protocol Buffers* (protobuf), a binary serialization format developed and open-sourced by Google. Serialization is the process of converting a data structure in memory into a byte sequence for transmission, and deserialization is the reverse.

The alternative most developers reach for is JSON, a human-readable text format. JSON has two disadvantages for a high-frequency internal bus. First, it carries no schema: any JSON object is syntactically valid, so a receiver cannot distinguish a malformed message from a valid one without application-level checks. Second, text encoding is inefficient: a 64-bit integer written as its decimal string representation takes 1–20 bytes; protobuf encodes the same value in 1–10 bytes using variable-length encoding.

Protobuf defines message types in `.proto` schema files, which are compiled to Rust code. A message that does not match the schema is rejected by the generated parser before it reaches application logic, catching schema violations at the boundary rather than deep inside a handler.

MessagePack was considered as an alternative: it is a binary JSON variant with compact encoding but no schema enforcement. *Cap'n Proto* and *FlatBuffers* offer zero-copy deserialization and faster parsing for large messages, but their Rust ecosystems are less mature than protobuf's. Protobuf has the best combination of schema enforcement, Rust library quality, and tooling.

Table 3 Inter-component communication paths

Path	Transport
Daemon ⟷ Daemon	Unix socket (protobuf)
App backend ⟷ Daemon	Unix socket (protobuf)
App backend ⟷ App frontend	Tauri command bridge (typed)
Compositor ⟷ Shell	Wayland protocols + Unix socket

A concrete example: a user action in the UI that queries the Knowledge Graph follows this path:

`UI event → App backend → Unix socket → Graph Daemon → Database`

Four steps, all local to the machine. The full round-trip is imperceptible to the user.

2.8.3 Knowledge Graph Rationale

A relational database such as *PostgreSQL* or *SQLite* organizes data into tables with rows and columns. Relationships between entities are expressed as foreign key references and queried using `JOIN` operations, which become progressively more expensive and harder to write as the number of hops between entities increases.

A *graph database* stores relationships as first-class data: entities are *nodes* and the connections between them are typed *edges*, both with their own properties. Traversing a relationship is a direct pointer follow, not a table scan or index lookup. Querying ”which files did the user work on last Tuesday, via which applications, and are any of those applications still running?” is a single graph traversal. In SQL it requires at minimum three joins, a time-range filter, and a correlated subquery for the running-process check.

For an operating system where everything is connected — processes open files, files belong to projects, projects span sessions, sessions involve multiple applications — a graph is the natural data model.

Kuzu is the specific graph database used here. It is an embedded graph database written in C++ with first-class Rust bindings, released under the MIT license. *Embedded* means it runs as a library inside the Graph Daemon process with no separate server process, no network port, and no additional service to manage. The database is a directory of files on disk that only the Graph Daemon process opens.

Neo4j was evaluated and rejected. Neo4j is the most widely deployed graph database and has a large ecosystem, but it runs as a separate server process, its community edition has usage restrictions, and its JVM-based runtime carries memory overhead inappropriate for a component embedded in a user session. SQLite with recursive Common Table Expressions for graph traversal was also considered; the query complexity and traversal performance for deep graphs made it unsuitable.

Kuzu uses *Cypher* as its query language, an open graph query standard (openCypher) originally developed for Neo4j. It reads close to natural language:

```
MATCH (f:File)-[:OPENED_BY]->(a:App)
WHERE f.last_modified > datetime('2026-03-01')
RETURN f.path, a.name
ORDER BY f.last_modified DESC
```

This query finds all files modified after a given date, which applications opened them, and returns the results sorted by modification time.

2.9 D-Bus Coexistence

D-Bus is the standard inter-process communication system on Linux desktops, in use since 2003. NetworkManager uses it to report network state changes, BlueZ uses it for Bluetooth events, `systemd-logind` uses it for session management, and the *XDG Desktop Portal* uses it to give sandboxed applications access to OS-level dialogs. It cannot be removed without breaking the wider Linux software ecosystem.

D-Bus works as a message broker: a central `dbus-daemon` process routes messages between senders and registered receivers identified by well-known bus names such as `org.freedesktop.NetworkManager`. This model works well for the ecosystem-level events it was designed for. The central broker and its overhead make it a poor fit for the high-frequency structured events between project-internal components, which is why the project uses its own Event Bus for internal communication.

This system does not replace D-Bus. It runs alongside it. D-Bus remains responsible for third-party application notifications via `org.freedesktop.Notifications`, hardware

events, portal interfaces, and session management. The internal Event Bus handles communication between project components only. The two systems are parallel and non-overlapping.

The rule for developers is simple: project-internal components communicate over the internal Event Bus via Unix sockets and protobuf. Standard Linux services and third-party applications communicate over D-Bus as they always have. No component spans both buses.

3 OS Foundation

Every operating system needs a foundation: a kernel that talks directly to hardware, an init system that starts services at boot, a package manager for installing software, and a base set of userspace tools. Building all of this from scratch would take decades and produce something inferior to what already exists. This project therefore builds on top of an existing Linux distribution that provides the entire foundation layer, and adds everything above it: the desktop environment, event system, knowledge graph, and AI layer.

3.1 What is a Linux Distribution

The *Linux kernel* is the core of the operating system: the program that manages hardware access, memory, processes, and the filesystem. It is developed by Linus Torvalds and thousands of contributors worldwide and is released under the GPL v2 license. The kernel alone, however, is not a usable system. A *Linux distribution* bundles the kernel together with an init system, a package manager, a C standard library, and a collection of userspace tools into a coherent, installable operating system.

Different distributions make different choices about package freshness, update cadence, governance, and default software selection. These choices have real consequences for a project building on top: a distribution with stale packages cannot support modern hardware or toolchains, and one with poor governance can change direction in ways that break downstream projects without warning.

3.2 Base Distribution

The base distribution has not yet been finalized; two candidates are under evaluation and the decision will be made before Phase 1 begins. Both use the *RPM* package format, which means the build pipeline and tooling are compatible with either choice.

RPM (RPM Package Manager) is one of the two dominant Linux package formats, alongside Debian's `.deb`. A package is a compressed archive containing the files to install, metadata about the package name and version, and dependency declarations listing which other packages must be present. The package manager resolves and installs these dependencies automatically.

The requirements for the base are deliberately narrow. Packages need to be current enough for modern development toolchains and recent hardware support, but

not so fresh that unvalidated changes arrive daily. Governance should not be fully concentrated in a single commercial entity. *Btrfs* must be available as a first-class filesystem, since the update strategy described in Section 3.3 depends on it. Driver and package coverage must be broad enough to support consumer hardware without manual intervention.

Candidate A: Fedora. Fedora [30] is a community distribution backed by Red Hat infrastructure but governed independently through the Fedora Council, which makes binding decisions on project direction without requiring Red Hat sign-off. Packages track upstream closely and a new release ships every six months. Btrfs has been the default filesystem since Fedora 33 (2020), and the Flatpak and Wayland ecosystems are better supported here than on most other distributions.

Table 4 Fedora: evaluation summary

Strengths	Weaknesses
Independent governance via Fedora Council	Six-month release cycle requires periodic major upgrades
Current packages, strong hardware support	Red Hat/IBM infrastructure in the background
Btrfs default since Fedora 33	Major upgrades need more integration testing than rolling updates
Strong Flatpak and Wayland support	

Candidate B: OpenSUSE Slowroll. OpenSUSE [62] offers three release variants. Tumbleweed is a fully rolling distribution that ships new packages daily. Leap follows a fixed annual release cycle tied to SUSE Linux Enterprise. *Slowroll* is a middle ground introduced in 2023: it pulls packages from Tumbleweed after a validation delay of several weeks, giving a cadence that is current enough for modern toolchains while providing a buffer against regressions. Btrfs and Snapper are more deeply integrated on OpenSUSE than anywhere else in the Linux ecosystem, which is a genuine advantage for the update strategy.

Table 5 OpenSUSE Slowroll: evaluation summary

Strengths	Weaknesses
Best Btrfs and Snapper integration in the ecosystem	SUSE owned by EQT Partners (private equity); governance risk
Rolling with validation delay; good update cadence	EQT ownership weakens European sovereignty narrative
Open Build Service is best-in-class for RPM packaging	Smaller community and hardware coverage than Fedora
European roots and presence	

The current preference is Fedora, primarily on governance grounds. The EQT ownership of SUSE is not a technical problem, but it is a liability when applying for European software independence funding through programmes such as the Sovereign Tech Fund or NGI: funders will scrutinize the ownership structure and a US private equity firm sitting above a nominally European project is a difficult position to defend. Fedora's governance is cleaner for that argument. Both distributions use RPM packaging, so the Open Build Service can target either without changes to the build pipeline.

3.3 Update Strategy

Updates on most operating systems involve some degree of risk: something breaks, the user does not know why, and getting back to a working state requires either expertise or luck. This system addresses that with a layered approach that makes the worst-case outcome a single keypress rather than an afternoon in recovery mode.

The approach is a mutable filesystem with automatic rollback protection. *Mutable* means the filesystem works the way Linux filesystems always have: software can be installed freely, system files can be modified, and standard developer tools work without restrictions. This distinguishes it from *immutable* distributions such as Fedora Silverblue and NixOS, where the root filesystem is read-only and changes require going through a specific toolchain. Immutable systems have genuine safety benefits, but they break workflows that the target audience relies on: installing language runtimes with `pip` or `cargo`, building projects from source, or running a local development server. The safety benefits of immutability are achievable through snapshot tooling without those costs.

Figure 3 shows the full update flow.

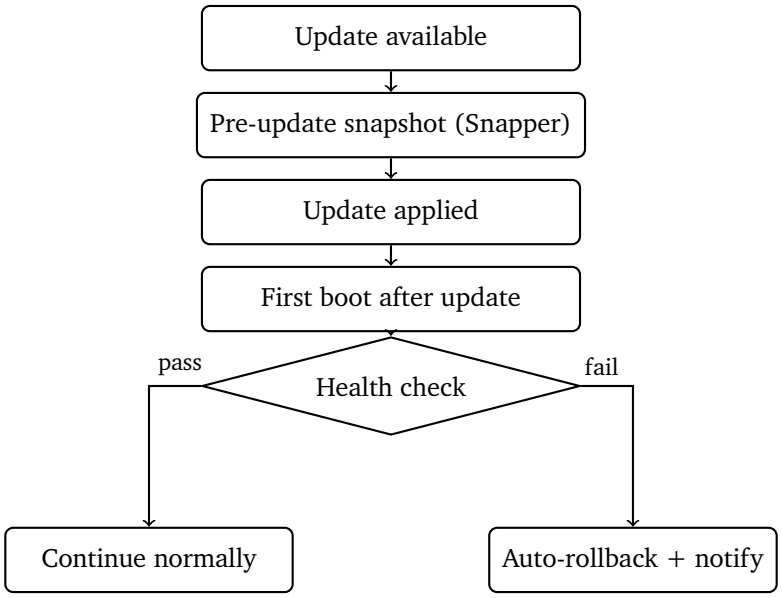

Figure 3 Update flow. A Snapper snapshot is created before every update. If the post-update health check fails, the system rolls back to the snapshot automatically and notifies the user.

Btrfs and copy-on-write. Most widely used Linux filesystems, including *ext4*, use *overwrite-in-place* semantics: when a file is modified, the old data is replaced at the same disk location. Once overwritten, the previous version is gone.

Btrfs [7] (B-Tree Filesystem) uses *copy-on-write* (CoW) semantics instead. When a file is modified, Btrfs writes the new data to a different location on disk and updates the metadata pointer. The old version remains intact until space is explicitly reclaimed. This has two important consequences: filesystem snapshots become nearly free in terms of disk space (a snapshot is a set of metadata pointers to existing blocks, not a copy of the data), and the filesystem remains self-consistent after a crash because incomplete writes never overwrite existing valid data.

Btrfs was merged into the Linux kernel in 2009 and has been the default filesystem on Fedora since 2020. Its CoW architecture is the technical foundation for this project's rollback strategy.

Btrfs subvolumes. A *subvolume* is a named, independently snapshotable region of a Btrfs filesystem. It behaves like a directory but can be mounted separately, snapshotted independently, and assigned its own quota limits. This project uses the following layout, consistent with the convention used by both Fedora and OpenSUSE:

```
@ -> mounted at / (system root)
@home -> mounted at /home (user data)
@snapshots -> mounted at /.snapshots
@log -> mounted at /var/log
@cache -> mounted at /var/cache
```

Keeping / and /home on separate subvolumes means a system rollback (restoring @ to a previous snapshot) never touches user data in @home. Log and cache data are excluded from system snapshots, preventing large log files from inflating snapshot sizes.

Snapper. *Snapper* [13] is the snapshot management tool that sits on top of Btrfs. It handles creating, listing, comparing, and restoring snapshots automatically. Snapper is configured per subvolume: the @ subvolume gets automatic pre/post snapshots around package manager operations and a timeline policy for background snapshots. The @home subvolume has its own independent daily snapshot schedule.

Before any update is applied, Snapper creates a tagged pre-update snapshot of the @ subvolume. This is a precondition for the update, not an optional step. A matching post-update snapshot is created after the package manager finishes, forming a before/after pair that Snapper can diff to show exactly what changed.

Bootloader integration. The bootloader is configured to list available snapshots as boot entries. If an update produces a system that fails to boot, the firmware presents the previous snapshot as a selectable entry on the next start. No terminal or recovery mode is required. On Fedora this is handled by grub2-snapper-plugin; on OpenSUSE the integration is built directly into the grub2 package and is one of the more polished aspects of the OpenSUSE experience.

Post-update health check. On the first boot after an update, a systemd oneshot service runs a health check before the desktop session starts. It verifies that the display system initializes, that the network stack is reachable, that the Graph Daemon and Event Bus start successfully, and that no systemd services have entered a failed state. If any check fails, the service triggers an automatic rollback by setting the pre-update snapshot as the default boot entry and rebooting. The user receives a plain-language notification on the rolled-back system.

Snapshot retention. Snapper's retention policy for this project removes snapshots older than seven days while always retaining the five most recent. Both values are user-adjustable in Settings. The `@home` subvolume retains the three most recent daily snapshots; home snapshots are a safety net for accidental file deletion, not a system recovery mechanism.

3.4 System Infrastructure

systemd. *systemd* [14] is the init system and service manager on both candidate distributions and on most modern Linux distributions. When the kernel finishes booting, the first userspace process it starts is `systemd` (PID 1), which reads *unit files* describing each service, its dependencies, and its startup environment, and starts them in dependency order. systemd also handles logging via the *journal*, socket activation, timer-based scheduling, and hardware event management.

All custom daemons in this project are deployed as systemd service units. systemd's unit file format provides sandboxing options that are enforced at the kernel level without any code changes in the service: `PrivateNetwork=true` to deny all network access, `PrivateTmp` for an isolated temporary directory, `MemoryDenyWriteExecute` to prevent writable executable memory, and `SystemCallFilter` to allowlist only the syscalls the service actually needs.

systemd-homed. *systemd-homed* [15] is a systemd component that manages user home directories as self-contained, portable, encrypted images. Unlike traditional Unix home directories, which are plain directories under `/home` identified by a numeric user ID, a systemd-homed home is a *LUKS-encrypted loopback image*: a file on disk that is mounted at login (after the encryption key is derived from the user's credentials) and unmounted at logout. When a user is not logged in, their home directory is an opaque encrypted file; even root cannot read its contents without the user's passphrase.

This is the technical foundation for the profile isolation system described in Section 8: each profile is a Linux system user with its own systemd-homed image. When a profile is not active, its image is unmounted and its key is discarded from memory.

LUKS (Linux Unified Key Setup [9]) is the standard disk encryption layer on Linux, built on top of the `dm-crypt` kernel module. It provides a standardized header format for encrypted block devices, support for multiple key slots so that several passphrases or key files can unlock the same volume, and memory-hard key derivation functions that make offline brute-force attacks expensive. LUKS is what systemd-homed uses internally to encrypt each home image.

Session manager and greeter. User sessions are managed by *greetd* [49], a minimal login daemon that handles authentication and session startup. greetd is intentionally small: it authenticates the user via PAM, launches the configured session command on success, and exits. All policy decisions, visual presentation, and user-facing interaction are delegated to a separate *greeter* process that communicates with greetd over a Unix socket using a simple JSON protocol.

This clean separation between authentication daemon and login UI is why greetd was chosen over display managers such as GDM or SDDM, which bundle both concerns into a single process with their own GUI frameworks. The login screen in this project is a first-party Tauri application that implements the greetd IPC protocol: it presents the user interface, collects credentials, and passes them to greetd for PAM verification. The Tauri greeter has access to nothing other than the greetd socket; it runs before any user session is active, with no access to the graph, the event bus, or any user data.

Package management. At the base level, `dnf` [69] is used on Fedora and `zypper` [63] on OpenSUSE. Both handle dependency resolution, cryptographic package signature verification, and repository management.

First-party project packages are built and distributed through the *Open Build Service* (OBS [59]). OBS is a hosted build infrastructure originally developed by SUSE and used by many open-source projects. Developers push package source files to OBS; OBS builds the resulting RPMs for every configured target distribution and architecture, signs them with the project GPG key, and hosts the repository. Building for both Fedora and OpenSUSE from the same spec files is one practical reason both candidates remain viable: the build pipeline does not change with the distribution choice.

Flatpak. Store applications are distributed as *Flatpak* [10] packages. Flatpak is a distribution-agnostic packaging format for desktop applications. A Flatpak bundles an application together with its specific library dependencies into a sandboxed unit that runs isolated from the rest of the system using Linux namespaces and *bubblewrap*, with controlled access to host resources via the *XDG Desktop Portal*.

The alternative formats are *Snap*, which routes all distribution through Canonical's servers and has faced criticism for this centralization, and *AppImage*, which is a self-contained executable with no sandboxing and no integrated update mechanism. Flatpak was chosen for its sandboxing model, distribution neutrality, and strong support in both Fedora and the broader Linux desktop ecosystem.

System-level components ship as native RPMs because they require tight integration with the system: daemons must run outside any sandbox, and systemd unit files must be installed to system paths. The Store handles both formats transparently.

3.5 Hardware Requirements

Three hard requirements determine the minimum supported hardware. They are not adjustable without significant changes to the security and graphics stack and are treated as hard constraints. In practice they exclude hardware older than roughly 2018–2019.

Table 6 Minimum hardware requirements

Requirement	Reason
Linux kernel 6.6 or newer	Required for the eBPF features used by the event monitoring system (Section 4.2) and for Intel CET hardware stack protection. Standard on both candidate distributions.
Vulkan-capable GPU	Required for hardware-accelerated compositing and for Windows compatibility via DXVK and vkd3d-proton. Covered automatically on AMD and Intel via Mesa; Nvidia requires the proprietary driver, with installer guidance provided.
systemd 255 or newer	Required for full systemd-homed portable home image functionality used by the profile system.

Vulkan. *Vulkan* [46] is a low-level graphics and compute API by the Khronos Group, designed as the modern replacement for OpenGL. It provides explicit control over the GPU at the cost of a more verbose API. This project requires Vulkan for two reasons: the Smithay compositor uses Vulkan for hardware-accelerated compositing, and the Windows compatibility layer (Section 8.3) translates Direct3D calls to Vulkan via DXVK and vkd3d-proton. On AMD and Intel hardware, Vulkan support is provided by *Mesa* [54], the open-source graphics driver stack included with both candidate distributions. Nvidia hardware requires Nvidia's proprietary driver because the open-source `nouveau` driver does not provide Vulkan support on most hardware generations.

3.6 Configuration Format

All system components and first-party applications use *TOML* [66] (Tom's Obvious Minimal Language) as the configuration format. TOML was designed by Tom Preston-Werner (co-founder of GitHub) to be unambiguous and easy to edit without special tooling.

The two most common alternatives are *YAML* and *JSON*. YAML is human-readable and widely used, but its indentation-sensitive syntax is a consistent source of subtle errors: a single misaligned space changes the meaning of the document, and certain values parse unexpectedly (bare `NO` is a boolean in YAML 1.1, not a string; YAML 1.2 fixes this, but library support for 1.2 is inconsistent in practice). JSON is unambiguous but has no comment support and is more verbose than necessary for human-edited files.

TOML has clear, explicit syntax with no indentation sensitivity, supports inline comments, and has an unambiguous type system: strings, integers, floats, booleans, datetimes, arrays, and tables. Using one format consistently means a developer who reads one configuration file can read all of them, and validation and migration tooling only needs to be written once.

Personal settings live at ~/.config/<app-id>/config.toml and system-wide settings at /etc/<service-name>/config.toml, following the *XDG Base Directory Specification* [34]. Some third-party applications expect other formats; a small helper translates without moving the source of truth:

```
# Export as environment variables for a legacy app
eval $(os-config-helper export \
    --toml ~/.config/myapp/config.toml --format env)

# Export as JSON for tools that require it
os-config-helper export \
    --toml ~/.config/myapp/config.toml --format json
```

The TOML file is always the authoritative source; the helper is a read-only translation layer with no business logic of its own.

4 Event System and Knowledge Graph

4.1 Overview

For the Knowledge Graph to build a picture of what is happening on the machine, something has to watch and report. The Event System is that something: it observes activity across the entire OS and feeds a continuous stream of structured events into the graph, without any visible effect on the user and without requiring applications to be rewritten for this platform.

Three sources feed into it, each covering a different slice of system activity.

Table 7 Event sources and their coverage

Source	App cooperation	What it sees
Kernel monitoring (eBPF)	None required	File opens, network connections, process lifecycle
Wayland compositor	Built-in (compositor is part of this project)	Window focus, app open/close, clipboard operations
First-party app events	Required; apps emit events directly	Rich semantic actions: document saved, search performed, tab opened

The design goal is baseline coverage for every app on the system without requiring anything from third-party developers, combined with richer semantic data where apps support it. eBPF provides the floor; structured app events raise the ceiling.

4.2 Kernel Monitoring with eBPF

eBPF (extended Berkeley Packet Filter) is a Linux kernel subsystem that allows small, sandboxed programs to run inside the kernel without modifying kernel source code or loading a kernel module [28]. The name is historical: the original Berkeley Packet Filter from 1992 was a mechanism for filtering network packets; the *extended* version, introduced in Linux 3.18 (2014) and substantially expanded in subsequent releases, is a general-purpose in-kernel execution environment with applications far beyond networking.

Before eBPF, the two main approaches to system monitoring were kernel modules and `ptrace`. A kernel module runs with full kernel privileges and no sandboxing; a bug in a module crashes the system. `ptrace` is a syscall that allows one process to intercept the syscalls of another, but it introduces substantial overhead and can only observe one process at a time. eBPF solves both problems: programs are verified by the kernel's eBPF verifier before loading (the verifier rejects any program that could loop infinitely, access out-of-bounds memory, or crash the kernel), run at near-native speed via JIT compilation, and can attach to hooks covering all processes on the system simultaneously.

An eBPF program attaches to a specific kernel hook, called a *tracepoint* or *kprobe*, and is executed each time the corresponding event fires: a file being opened (`openat` syscall), a new process being created (`clone`/`execve`), a network connection being established, and so on. The program runs in kernel context, collects the relevant data (file path, process ID, return value), and passes it to a user-space daemon via a shared *ring buffer* without the overhead of standard kernel I/O paths. The result is a read-only window into every process on the machine with overhead typically below 2% of CPU time under normal desktop load.

At this layer, the system knows *that* a process opened a file but not *why* or what it means to the user. That is intentional: eBPF provides the universal baseline, and higher-level sources fill in the semantics where they are available.

Why aya. Three libraries exist for writing eBPF programs from user space: `libbpf` [50], `BCC` (BPF Compiler Collection), and `aya` [21].

`libbpf` is the reference C library maintained in the Linux kernel tree. It is stable and widely used, but integrating it into a Rust codebase requires C FFI (Foreign Function Interface) bindings. Every call across the Rust/C boundary requires `unsafe` blocks that the Rust compiler cannot verify, partially defeating the purpose of writing the daemon in Rust.

`BCC` takes a different approach: eBPF programs are written as C strings embedded in Python or Lua and compiled at runtime by LLVM on the target machine. This makes BCC inappropriate for a production daemon: it requires LLVM to be present on the user's system, compilation adds startup latency of several seconds, and the Python runtime overhead is significant for a background service.

`aya` is a Rust-native eBPF library [21]. Both the eBPF programs (which run inside the kernel) and the user-space daemon are written in Rust. Programs are compiled to eBPF bytecode at build time using `cargo`, not at runtime, so no compiler is required

on the end-user's machine. The result is a fully Rust codebase with no unsafe FFI boundaries, no runtime compilation, and no additional system dependencies beyond what the kernel already provides.

Noise filtering. eBPF can generate thousands of events per second under normal load, most of which carry no useful information for the Knowledge Graph. Two filters in the normalizer reduce this before events reach the bus. Identical operations on the same file by the same process within 100 milliseconds are collapsed into a single event with an updated timestamp. Paths under `/proc`, `/sys`, `/dev`, `/tmp`, and shared libraries under `/usr/lib` are discarded entirely, since they have no semantic value in a graph of user activity.

4.3 Event Bus

The Event Bus is a background daemon that receives events from all sources, validates them against the event schema, and forwards them to registered consumers. Its only job is to move and validate data; it does not interpret or store anything. Figure 4 shows the full pipeline.

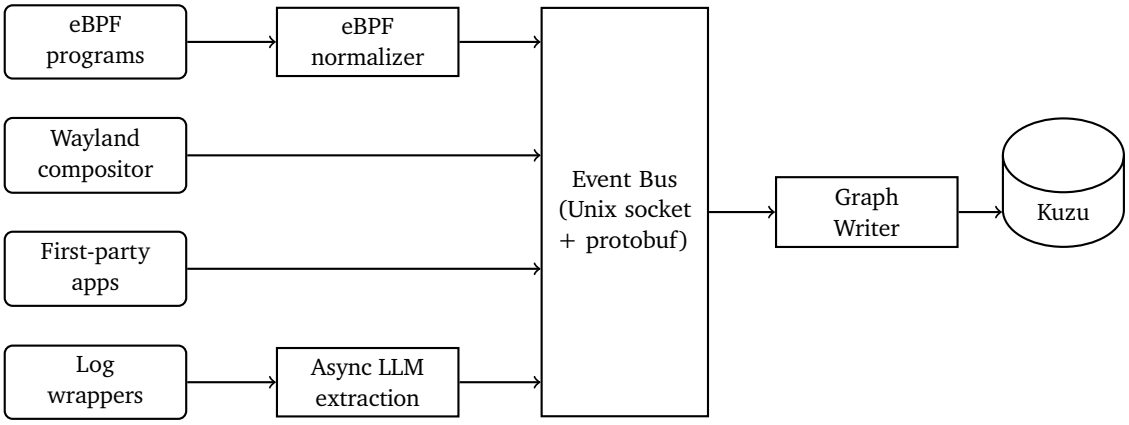

Figure 4 Event pipeline. Raw kernel events pass through a normalizer before reaching the bus. Log output from third-party applications is processed asynchronously by an LLM that extracts structured entities. All sources converge on the Event Bus, which validates and forwards to the Graph Writer Daemon.

Event schema. Every event, regardless of source, shares a common envelope. The identifier uses *UUID v7* [25], a variant of the universally unique identifier standard introduced in RFC 9562 (2024). Unlike UUID v4, which is purely random, UUID v7 embeds a millisecond-precision Unix timestamp in the most significant bits. This makes UUIDs generated close together sort together lexicographically, which matters for a time-series workload: inserting events into the graph in roughly time order avoids the random-access write pattern that UUID v4 would produce and improves query performance for time-range lookups. The envelope also carries a microsecond

timestamp, a type label, a source identifier, the originating process ID, and a session reference. The type-specific data travels as a protobuf payload inside the envelope.

```
message Event {
  string id = 1; // UUID v7 (time-sortable)
  string type = 2; // "file.opened", "window.focused", etc.
  int64 timestamp = 3; // microseconds since epoch
  string source = 4; // "ebpf", "wayland", "app:<id>"
  uint32 pid = 5; // originating process
  string session_id = 6; // user session reference
  bytes payload = 7; // type-specific protobuf message
}
```

Listing 1 Common event envelope (protobuf)

A file.opened payload carries the file path, the application name, and the operation flags. A window.focused payload carries the app ID and window title. App events use a generic structure with a category, an action, a subject, and an optional metadata map, which allows any first-party application to emit meaningful events without requiring a dedicated payload type per application.

Clipboard content is never logged. The bus records that a clipboard operation occurred and its MIME type, not the content itself.

D-Bus coexistence. The custom Event Bus handles communication between project-internal components only. D-Bus remains in place for everything it already handles: desktop notifications, NetworkManager, Bluetooth, session management via systemd-logind, sandboxed app portals, and standard third-party applications. The two systems run in parallel with no overlap; nothing that communicates over D-Bus today is changed.

4.4 Backpressure and Batching

Kuzu is an embedded graph database optimized for analytical queries over large graphs, not for high-frequency single-row writes. Writing each eBPF event individually as it arrives would saturate the write path quickly. Instead, the Graph Writer Daemon maintains a ring buffer and writes to Kuzu in batches.

> **Phase 0 validation required.** The batching architecture below is designed to make Kuzu viable under sustained eBPF load. This must be confirmed with a benchmark before the architecture is finalized: Event Bus plus Graph Writer under simulated load targeting 5,000 events per second, with Kuzu as the write target. If Kuzu cannot sustain this load even with batching, the fallback is a two-layer architecture with SQLite handling writes and Kuzu receiving promoted data periodically for queries. This is a Go/No-Go decision at Phase 0.

The ring buffer holds 10,000 event slots. A batch write is triggered by whichever threshold is reached first: 500 milliseconds have elapsed, or 1,000 events have accumulated. Both values are configurable. When the buffer is full, the system works through three tiers before discarding anything. First, it checks whether an identical

event (same type, source, and subject) already exists in the buffer and, if so, updates its timestamp rather than inserting a duplicate. If no duplicate exists, it drops the lowest-value event currently in the buffer to make room, prioritizing raw eBPF read and write events with no app-level context. Only if neither approach frees a slot is the incoming event discarded, which should not occur under normal operating conditions. Drop rates per tier are exposed as metrics and monitored by the Anomaly Detector.

4.5 Knowledge Graph

Why a graph. A relational database stores relationships as foreign keys across tables. A graph database stores them as first-class edges. The difference is felt most when querying across multiple entity types simultaneously: finding which files a user worked on last Tuesday, which applications touched them, and whether any of those applications are still running requires three joins and a subquery in SQL and a single traversal in a graph. For an operating system where everything is connected (processes open files, files belong to projects, projects span sessions, sessions overlap with other applications), a graph models the domain naturally.

Kuzu. The Knowledge Graph is implemented with *Kuzu* [48], an embedded graph database written in C++ with first-class Rust bindings, released under the MIT license. Embedded means it runs inside the Graph Daemon process with no separate server process, no network socket, and no additional service to manage. The database is a directory of files on disk that only the Graph Daemon process opens directly.

Kuzu uses *Cypher* as its query language [61]. Cypher is a declarative graph query language originally developed by Neo4j and now maintained as an open standard under the openCypher project. It reads close to natural language: nodes are written as (n:Label), relationships as -[:TYPE]->, and the query describes the pattern to match rather than the steps to execute. Anyone familiar with SQL can read Cypher within minutes; anyone who has used Neo4j will find it identical.

```
MATCH (f:File)-[:OPENED_BY]->(a:App)
WHERE f.last_modified > datetime('2026-03-01')
RETURN f.path, a.name
ORDER BY f.last_modified DESC
```

Listing 2 Example Cypher query: files accessed last Tuesday

Schema. Figure 5 shows the core node types and their relationships. The schema is intentionally minimal at this stage; it will grow as the system matures. File, App, Session, Project, Event, and UserAction are the primary entities. Every raw occurrence from the event pipeline lands as an Event node first; higher-level UserAction nodes are derived from clusters of events, potentially with LLM assistance, and represent meaningful activity rather than individual syscalls.

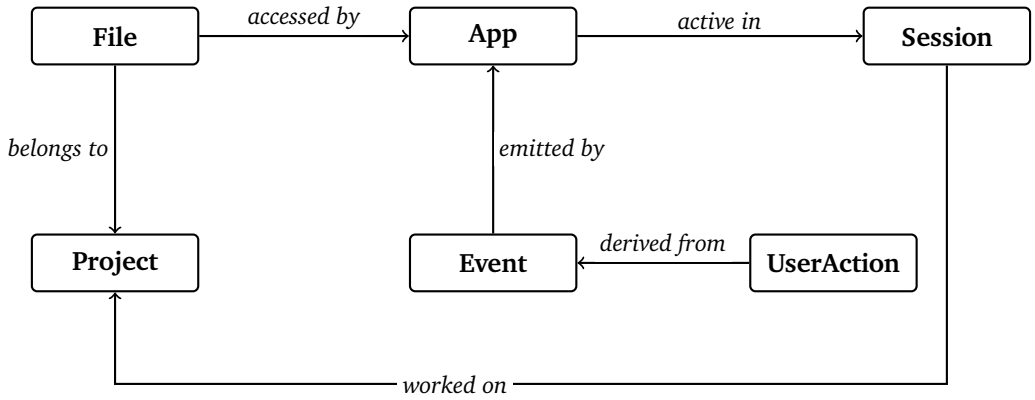

Figure 5 Knowledge Graph core schema. Nodes represent entities; edges represent relationships. `Event` nodes are the lowest-level entries, created directly from the event pipeline. `UserAction` nodes are derived abstractions built from clusters of raw events.

4.6 Retention Policy

The graph accumulates data indefinitely if left unchecked, which creates two problems: storage grows, and the privacy surface area expands. The retention policy defines how long data lives, with user control at every tier.

Table 8 Knowledge Graph retention tiers

Tier	Default	Content
Raw eBPF events	30 days	High-volume, low-semantic-value syscall events. Already incorporated into higher-level nodes after ingestion; the raw records are largely redundant beyond the anomaly detection window.
Semantic nodes	12 months rolling	File, App, Session, and NetworkConnection nodes representing actual activity. This is where AI query value lives. Nodes past the window are summarized rather than deleted.
Pinned nodes	Permanent	Nodes explicitly marked by the user, active Project nodes, and nodes recently referenced in AI queries. Only manual deletion removes these.

Summarization. Hard deletion of aged nodes loses information abruptly. Instead, nodes past the retention window are compacted into summary nodes that retain aggregate statistics (access count, primary application, active period) while discarding the individual event edges. The node still exists in the graph and AI queries still find

it; the granularity is reduced but the semantic content is preserved. Summarization is triggered by access frequency rather than age: a node that has not been queried in six months is a candidate regardless of when it was created, while a node from two years ago that is queried regularly remains granular. A background daemon identifies candidates nightly and processes them during idle periods.

User controls. Settings exposes the retention window per tier with documented minimums, a manual summarize-now option for specific time ranges, node and subgraph deletion, full graph export for data portability, and storage usage broken down by tier and time range. In managed deployments, administrators can set a minimum retention for compliance-relevant audit events in a separate audit log, but they cannot shorten the user's configured retention window for personal graph data.

4.7 Temporal Filesystem View

The Knowledge Graph contains a complete record of which files were accessed, created, and modified, when, and by which application. This makes it possible to answer queries like "show me everything I worked on last Tuesday" without relying on file modification timestamps, which applications regularly reset, or on search indices, which do not capture access history.

The temporal filesystem view exposes this as a navigable virtual directory tree via a *FUSE* (Filesystem in Userspace) mount. FUSE is a Linux kernel mechanism that allows a userspace process to implement a filesystem: the kernel forwards directory listing and file open calls to the FUSE daemon, which answers them from its own data source rather than from disk. The temporal view mounts at `~/.timeline/` and is populated on demand by a small daemon that translates directory traversal into Graph Daemon queries.

A path like `~/.timeline/2026-03-04/` lists all files the user touched on that date, grouped by project where the graph has enough context to determine project membership. Opening a file through this path opens the actual file at its real location; the timeline view is an index, not a copy. No additional storage is used.

```
# List files worked on a specific date
ls ~/.timeline/2026-03-04/

# Find all files touched in the last week, by project
ls ~/.timeline/last-7-days/

# Open a file as it appears in the timeline (opens real file)
xdg-open ~/.timeline/2026-03-04/ProjectAlpha/report-draft.md
```

Listing 3 Temporal filesystem view examples

The FUSE daemon holds no persistent state of its own; every listing is a live graph query. The mount is available only while the user session is active and the Graph Daemon is running. It is read-only: files cannot be created, renamed, or deleted through the timeline path.

A natural extension of the temporal view is causal chain visualization: because the graph records not only which files were accessed but the sequence and application context of those accesses, it is possible to reconstruct the workflow that produced a given output. A query asking ”what did I do to produce this file?” can trace the chain of files opened, commands run, and applications active in the session where the file was last modified. This is planned as a Phase 1 feature built on the same FUSE and graph infrastructure described here.

Automatic project inference. Project nodes in the graph are created explicitly when the user names a project, but the graph also accumulates enough signal to infer project membership automatically. Files that consistently appear together in the same sessions, terminal working directories that recur alongside a fixed set of applications, and browser tabs that co-occur with specific local files are all indicators that a coherent project exists even without a name. A background inference pass groups these co-occurrence clusters into candidate project nodes and presents them to the user as suggestions rather than applying them automatically. The user can confirm, rename, or dismiss each suggestion. Automatic inference is opt-in and runs locally.

5 AI Layer

5.1 Overview

The AI layer sits on top of the Knowledge Graph and provides both the user and the system with access to an AI model that has structured context about what is happening on the machine. It is not a chatbot grafted onto an existing OS; it is an interface that can query the graph, invoke application capabilities through a standardized protocol, and optionally act autonomously when the user explicitly enables that.

The AI layer is not the primary reason this system exists. The desktop, the permission system, the profile isolation, and the store all stand on their own. AI is an enhancement that becomes more useful as the Knowledge Graph fills in over time, and users who want none of it can disable it completely without losing any other functionality.

Three principles govern the design of this layer.

Query-first. The AI responds when asked. Nothing happens in the background without an explicit trigger. Autonomous features exist but are separate opt-ins, not defaults that the user has to find and disable.

Provider-agnostic. The system does not care whether the model runs locally or in the cloud. Every provider implements the same interface; the rest of the codebase never speaks to a specific backend. The user decides what runs where, including routing sensitive requests to a local model only.

Opt-out at any point. The entire layer can be disabled in Settings under AI. When disabled, no local model runs, no graph queries originate from the AI layer, Spotlight

falls back to standard search, and no MCP servers are active. The setting is not hidden five menus deep.

5.2 Cold-Start Behavior

The Knowledge Graph is empty on a fresh install. An AI feature that promises contextual awareness but has nothing in the graph yet is worse than no feature at all: it creates an expectation and immediately fails to meet it. The system handles this with straightforward communication rather than by faking usefulness.

During onboarding, AI features are presented with an honest explanation of what to expect: the assistant learns context gradually as the system is used, starts with nothing, and becomes more useful over days and weeks. There are no demo interactions, no placeholder context, and no fabricated examples of what it will eventually be able to do.

At onboarding the user can optionally seed the graph with existing data from their current system: browser history, calendar events, and documents. This is presented as a trade-off, not a recommendation. Each source can be imported independently, so someone who is comfortable sharing their calendar but not their browsing history can import only the former. Importing gives the AI immediately useful context without a multi-week waiting period.

The AI assistant does not surface proactively until a threshold is met: seven days of usage or a completed manual import. Before that threshold the feature is accessible but does not advertise itself. The non-AI parts of the system are functional and complete from the first boot, so there is no gap in usefulness during this period.

5.3 Provider Abstraction

Different AI providers have different APIs, different latency profiles, and significantly different privacy implications. Sending the contents of a private file to a cloud API is appropriate in some contexts and completely unacceptable in others. The system needs to handle both without the rest of the codebase caring which provider is active.

The solution is a Rust trait that every AI backend implements. The AI layer constructs a `CompletionRequest`, passes it to whichever provider is configured, and receives a `CompletionResponse`. No provider-specific code is visible outside the provider module. The `#[async_trait]` macro is a Rust community crate that enables async methods in traits, which the stable Rust compiler does not yet support natively; it desugars the async methods into an equivalent form that compiles correctly.

```rust
#[async_trait]
pub trait AiProvider: Send + Sync {
    async fn complete(
        &self,
        request: CompletionRequest,
    ) -> Result<CompletionResponse>;
    async fn available(&self) -> bool;
    fn name(&self) -> &str;
}
```

Listing 4 AI provider trait

Figure 6 shows how the AI layer relates to the available backends.

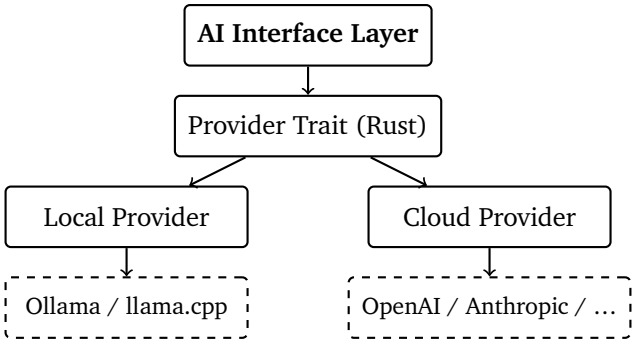

Figure 6 Provider abstraction. The AI layer speaks only to the trait; the concrete backend is a configuration choice. Local and cloud providers are interchangeable from the perspective of anything above the trait boundary.

Local providers. A local provider runs an AI model entirely on the user's machine. No data leaves the device. The two primary options are *Ollama* [58] and *llama.cpp* [36].

llama.cpp is a C++ inference engine for running large language models on consumer hardware, originally developed by Georgi Gerganov in 2023. It achieves viability on consumer hardware through *quantization*: a technique that reduces the numerical precision of a model's weight values (for example from 32-bit floats to 4-bit integers), shrinking both the memory footprint and computation cost at a modest quality trade-off. A 7-billion parameter model that would require roughly 28 GB of RAM at full 32-bit precision runs in approximately 4 GB after 4-bit quantization, which falls within the RAM budget of a typical consumer laptop. llama.cpp exposes an HTTP API compatible with the OpenAI request format, so any client written for the OpenAI API works against a local llama.cpp instance by changing the base URL.

Ollama wraps llama.cpp and other backends in a higher-level tool that adds model management (download, version tracking, storage), a simpler CLI, and a REST API. For most users, Ollama is the practical entry point. llama.cpp is available as a direct option for cases where fine-grained control over quantization levels and inference parameters is needed.

Both are supported as local provider backends. The specific model is a user configuration choice. As of 2025, models in the 7B–13B parameter range with 4-bit quantization offer the best balance of quality, memory footprint, and inference speed on consumer hardware without a dedicated GPU.

Cloud providers. Cloud providers send the request to a remote API over HTTPS and return the response. The system ships with built-in support for the OpenAI [60] and Anthropic [1] APIs, both of which follow similar request/response formats. Additional providers can be added by implementing the provider trait; the interface is intention-

ally minimal. API keys are stored in the system keyring (not in a config file) and are never written to the Knowledge Graph.

Routing policy. Multiple providers can be active simultaneously under a routing policy the user configures in TOML. The baseline: use the local provider if one is configured and available; fall back to cloud if the local model is unavailable or if the request exceeds the local model's context window. Additional rules can be layered on top: route all code-related requests to a specific provider; prevent any request that includes personal file contents or graph data from leaving the machine; route requests originating from a specific application to a designated provider. Rules are evaluated in order; the first match wins.

5.4 MCP Integration

MCP (Model Context Protocol) is an open protocol published by Anthropic in November 2024 that standardizes how AI models interact with external tools and data sources [2]. The core idea is a client/server split: the AI model talks to an *MCP client*, which in turn dispatches to one or more *MCP servers*. Each server exposes a list of *tools* (functions the AI can call) and *resources* (data the AI can read). The protocol runs over JSON-RPC 2.0, a lightweight remote procedure call format, typically over a Unix socket or stdio pipe for local servers and over HTTP/SSE for remote ones.

The key property of MCP is that the AI model does not need to know anything about the application it is talking to. A model that has never seen this OS can still call `list_directory` on the file manager MCP server, because the tool definition (name, parameters, description) is discovered at runtime. This makes MCP servers independently deployable and the AI layer application-agnostic.

In this system, first-party applications expose MCP servers as part of their standard implementation via the *os-sdk* boilerplate described in Section 2: the file manager exposes `list_directory`, `open_file`, and `move_file`; the terminal exposes `run_command` and `get_output`; the Knowledge Graph exposes a query interface. The AI can coordinate across multiple applications in a single interaction by calling their respective servers in sequence. Figure 7 shows the topology.

CLI tools do not need MCP servers. If a tool has a `–help` flag and produces sensible output, the AI can invoke it as a subprocess and read `stdout`. MCP is reserved for GUI applications and stateful interfaces where a subprocess call is insufficient.

5.5 Interaction Model

Phase 1: Query-based (default). The user asks a question, the AI responds. No background activity occurs without an explicit trigger. The AI has access to the Knowledge Graph and whatever MCP servers are permitted under the current configuration. Typical interactions include querying session history, finding files related to a project, correlating system events with observed behaviour, and composing multi-step actions across applications.

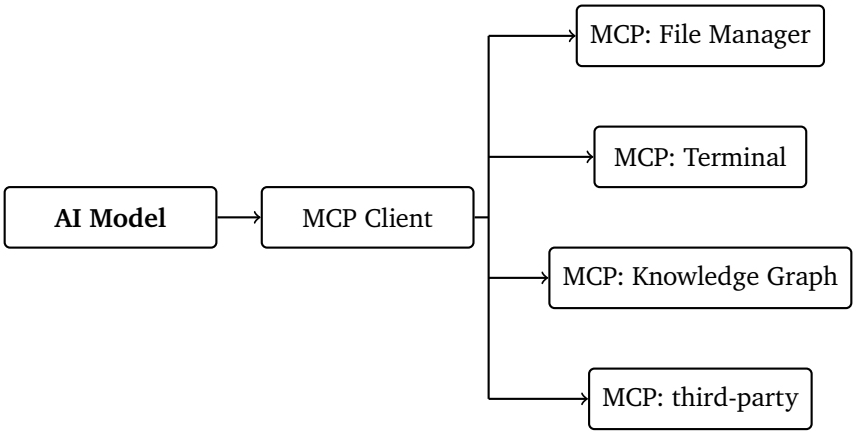

Figure 7 MCP topology. The AI model communicates through a single MCP client that dispatches to individual application servers. Third-party applications from the Store can ship their own MCP servers, subject to the permission model described in Section 5.6.

When a question is asked, the AI layer translates the natural language input into a Cypher query, executes it against the Knowledge Graph via the Graph Daemon, and passes the structured results to the model as context. Relative time expressions are resolved to absolute UTC timestamps before the generation step, so the model always operates on concrete values rather than ambiguous phrases. The Cypher generation and the result formatting are two separate model calls, since combining them causes the model to bias the query toward a desired-sounding answer rather than accurately querying the schema.

The query validator sits between generation and execution: it checks that the generated Cypher uses only known node types and relations, contains no write operations, and includes a `LIMIT` clause. If validation fails, the error is fed back for a retry; after two failed attempts the user is told the question could not be answered, with no silent fallback.

Phase 2: Autonomous agent (explicit opt-in). A background agent that observes the event stream and acts proactively is available as an opt-in capability. It is not enabled by default and each autonomous behaviour is a separate toggle: suggesting related files when a document is opened is independent of automatically tagging new files by project. The agent uses the same provider abstraction and MCP layer as the query interface; the difference is that it initiates actions rather than responding to them.

5.6 MCP Permission Model

MCP servers are the interface through which the AI acts on applications. The permission model distinguishes between reading information, which the AI can do by default within the user's configured AI permission level, and modifying state, which requires explicit per-session authorization.

Table 9 Read-only MCP servers (permitted by default)

Server	Exposes
File Manager	Directory listings, file metadata
Calendar	Upcoming and past events
Browser	History and open tabs (no passwords, no form data)
System Monitor	Process list and resource usage
Notes	Note titles and content

Reading to answer a question is what the user expects when they ask the AI something. Answering "what meetings do I have today?" without a confirmation dialog is the correct behaviour. Reading does not change state, and the AI permission level configured in Settings still gates what data each server exposes regardless of this default.

Table 10 Write and action MCP servers (per-session authorization required)

Server	Can do
Terminal	Execute shell commands
File Manager	Create, move, rename, delete files
Calendar	Create and modify events
Email	Compose and send messages
Store	Install and remove packages

Authorization is granted once per session and covers the scope the user specifies: approving file management does not imply approval for email. It expires at session end and is not persisted. Within an authorized scope, a separate hardcoded list of actions always requires explicit confirmation regardless of session permissions, because their consequences are irreversible or high-impact: permanent file deletion, sending any external message, installing or removing system packages, modifying system configuration outside ~/.config, any network request to a host outside the application's declared permissions, and running commands with elevated privileges. This list is not configurable.

Third-party MCP servers shipped by Store applications are always treated as action-requiring by default. A community server that claims to be read-only receives the same treatment until it carries a verified security audit covering its MCP implementation.

5.7 System Explanation Mode

A running operating system is largely opaque to its user. Background processes run, network connections open and close, disk activity spikes, and the user has no practical way to understand what is happening without opening a terminal and running several diagnostic commands. This is a gap this system is positioned to close, because it already has the data.

The AI daemon can answer the question *"What is my computer doing right now?"* by querying the live event stream and the Knowledge Graph simultaneously. The event stream provides the current moment: which processes are active, what they are reading or writing, which network connections are open, what the CPU and memory usage is and which process owns it. The Knowledge Graph provides context: whether this activity is normal for this time of day on this machine, which project the files being accessed belong to, whether this process has behaved unusually in the past.

The result is a dynamically generated plain-language summary, not a set of predefined status strings. A typical response might note that the package manager is fetching updates in the background, that a first-party application is indexing a recently imported folder, and that network activity is within the normal range for this time of day. If something is unusual, the summary says so: a process accessing node types it has never previously accessed, or a network connection to a destination outside the application's declared permissions, is flagged in the same natural language response rather than silently logged.

The explanation mode is available on demand via the AI query interface and as a persistent ambient view in System Monitor. It does not run continuously in the background; it is a query like any other, executed when the user asks. Everything it uses is local: the event stream and the Knowledge Graph never leave the machine, and the model generating the summary runs locally unless the user has configured a cloud provider. The output can be shared by the user if they choose, but nothing is reported automatically.

6 Desktop Environment and Design Language

The desktop environment is the most visible part of the system and the primary differentiator for users switching from other platforms. The goal is a fast, coherent desktop that does not feel like a collection of independent projects assembled by different teams in different decades. The stack reflects this: performance-critical and system-close code is Rust, everything visual is TypeScript with Tailwind CSS, and the two halves communicate over well-defined interfaces that neither side reaches past.

6.1 Architecture and IPC

Three communication channels connect the compositor core to the shell and its frontend, each carrying a different class of data.

Wayland protocol extensions are the correct channel for anything the Wayland ecosystem has standardized. The Unix socket exists for everything that does not have a protocol equivalent and where a custom solution is both simpler and more direct. Messages on the socket use JSON during development, with migration to protobuf planned once the schema stabilizes.

Within the Tauri shell, the Rust backend and TypeScript frontend communicate through Tauri's typed command system: the frontend calls Rust via `invoke()`, Rust

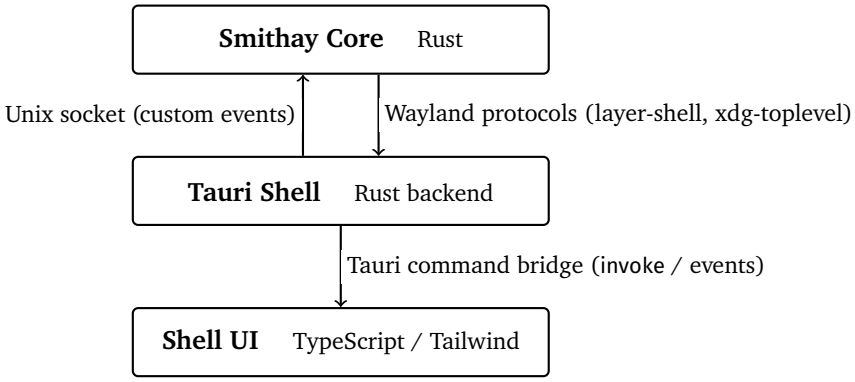

Figure 8 Shell IPC architecture. Wayland protocols carry window management data (open windows, focus, workspace state). The Unix socket carries system-specific events with no Wayland equivalent: theme updates, module lifecycle events, graph notifications. The Tauri command bridge connects the Rust backend to the TypeScript frontend within the shell process.

pushes events to the frontend via Tauri's event emitter. No ad-hoc serialization; the bridge handles it.

6.2 Compositor and Window Manager

The Wayland compositor is built on *Smithay* [72] (introduced in Section 2.6). Smithay handles three distinct layers of low-level plumbing that would otherwise require months of work per area.

DRM/KMS (Direct Rendering Manager / Kernel Mode Setting) is the Linux kernel subsystem responsible for talking directly to display hardware. DRM manages GPU memory and command submission; KMS handles setting display modes, resolutions, refresh rates, and which GPU output drives which physical connector. A compositor that wants to draw to the screen without going through another display server must use DRM/KMS directly. Smithay abstracts this into a safe Rust API [8].

libinput [51] is the standard Linux input processing library. It reads raw events from kernel input devices (keyboards, mice, touchpads, touchscreens), applies device-specific quirks and acceleration curves, and emits normalized events. Using libinput means the compositor gets correct two-finger scrolling, palm rejection, and tap-to-click on every supported touchpad without implementing any of that per-device logic itself.

Buffer management in Wayland means handling *wl_buffer* objects: the shared memory regions or GPU buffers that applications render their window contents into. The compositor receives these buffers, composites them together (applying transforms, clipping, and blending), and presents the result to the display hardware via DRM/KMS. Smithay handles the buffer lifecycle including GPU-side zero-copy paths via `dmabuf`.

Compositor logic built on top of Smithay: window placement, focus rules, and workspace behaviour are all written by this project. Smithay does not prescribe any of it. The *COSMIC* desktop compositor from System76 (`cosmic-comp` [74]) is a useful reference for a production Rust-first Smithay compositor with comparable goals.

Window management supports both floating and tiling layouts, switchable per workspace independently.

Wayland protocol extensions. The core Wayland protocol covers only the basics: surfaces (drawable regions), input routing, and output (monitor) management. Everything a desktop actually needs – panels anchored to screen edges, a taskbar that knows about open windows, workspace switching – requires *protocol extensions*: additional interface definitions that the compositor implements and clients can use if present. Table 11 lists the extensions this compositor implements and what each enables.

The wlr- prefixed protocols are part of the *wlroots* protocol collection [90], originally developed alongside the wlroots compositor library and now widely implemented across compositors including Sway, Hyprland, and KWin.

XWayland. X11 compatibility is provided via *XWayland* [20], an X11 server that runs as a Wayland client. From the compositor's perspective it is just another Wayland client; from legacy X11 applications' perspective it is a regular X11 display. It starts on demand when the first X11 application launches and shuts down when none are running. Users who do not need X11 compatibility can disable it in Settings.

XWayland has structural limitations that cannot be solved at the compositor level; they are documented rather than hidden. Copy-paste between X11 and Wayland applications requires a clipboard synchronization bridge. Drag-and-drop across protocol boundaries is unreliable. X11 applications cannot use xdg-desktop-portal for screen-sharing. Most significantly, X11 has no inter-application isolation: one X11 application can read the input events of any other X11 application within the XWayland session, which is a keylogger-class capability contained within the XWayland boundary. This is recorded in the audit log whenever X11 applications are active.

6.3 Shell and Application Framework

The shell – taskbar, app launcher, notification center, quick settings panel, and lock screen – is built with Tauri (introduced in Section 2.5) using TypeScript and *Tailwind CSS* for the interface layer. Tauri renders the UI in a *WebKitGTK* WebView, which provides full modern web rendering capabilities without the memory cost of an Electron-style approach. The shell is a Wayland client that uses wlr-layer-shell to anchor its surfaces to screen edges.

What GTK is. *GTK* (GIMP Toolkit, now branded simply as GTK) is the most widely used UI toolkit on Linux [38]. It provides the standard set of interface components – buttons, text inputs, dialogs, menus, scrollable lists – and is the foundation for applications such as GNOME, Nautilus, and most of the Linux application ecosystem. *GTK4* is the current major version, a significant rewrite from GTK3 with a new rendering pipeline (using GPU shaders via OpenGL/Vulkan rather than Cairo), improved accessibility, and a better layout engine.

Qt is the main alternative to GTK and is used by the KDE ecosystem and many cross-platform applications [78]. The first-party applications in this project do not use GTK

Table 11 Implemented Wayland protocol extensions

Protocol	What it enables
wlr-layer-shell [90]	Allows a client (the taskbar) to claim a fixed strip at a screen edge and stay above all application windows. Without this, the taskbar would be just another window the user could accidentally cover.
wlr-foreign-toplevel-management [90]	Lets a client outside the compositor query and control all open windows: title, app ID, minimized/maximized state, and focus. This is what makes the taskbar able to list running applications and switch to them.
wlr-workspace-management [90]	Exposes workspace state (current workspace, window-to-workspace mapping) to clients. Enables the workspace switcher in the taskbar.
xdg-shell	The standard protocol for normal application windows: creation, resize, maximize, fullscreen, and popup menus. Every Wayland application uses this.
xdg-output	Extends monitor information beyond the core protocol: logical position and size for multi-monitor layouts, human-readable names.
xdg-desktop-portal [35]	A D-Bus interface (not a Wayland protocol) that gives sandboxed applications controlled access to OS capabilities: file pickers, screensharing, printing, and camera access, each with a user-consent dialog. Flatpak applications use this exclusively; they have no direct filesystem or device access.
wp-fractional-scale-v1	Allows the compositor to tell an application its exact fractional scale factor (for example 1.5 on a 1440p display), so it can render at the correct pixel density rather than relying on integer scaling with blurring.

or Qt directly; they use Tauri with WebKitGTK. However, GTK4 and Qt applications from the ecosystem still run on this desktop and receive theming coverage, which is why both frameworks appear in the theming section.

Tailwind CSS. *Tailwind CSS* [76] is a utility-first CSS framework. In conventional CSS, developers write named classes that each describe a component (`.notification-card { padding: 1rem; border-radius: 0.5rem; }`). In Tailwind, styling is applied directly with utility classes in the HTML: `<div class="p-4 rounded-lg bg-surface">`. Each class does exactly one thing. The result is that the visual design and the structure live together in the component file; there is no separate stylesheet to maintain or keep in sync.

For a project where multiple developers will build different applications, this matters: consistency comes from shared utility classes and design tokens, not from trusting that each developer remembers to use the right custom class name. A button looks the same across applications because it uses the same token values, not because two people independently wrote matching CSS.

Tailwind is a build-time tool. Only the utility classes actually used in the project are included in the final CSS output; unused classes are removed.

ui-kit. A shared TypeScript package provides the visual foundation for every first-party application. It is based on *shadcn/ui* [71], a collection of accessible, unstyled UI components built on *Radix UI* [91] primitives and styled with Tailwind.

The key design decision behind shadcn/ui is its distribution model: components are not installed as an npm dependency. Instead, component source files are *copied into the project* and become part of the codebase. This means there is no upstream version to break things silently, no peer-dependency conflicts, and full ownership of every component. When a component needs to deviate from the default (different animation, extra accessibility attribute), the source file is there to edit directly.

ui-kit is organized in three layers. The first layer is the *design token system*: values defined as CSS custom properties that establish the entire visual language. A *CSS custom property* (also called a CSS variable) is a named value defined in a CSS rule and referenced throughout the stylesheet: `--color-accent: #3b82f6` defined once and used as `color: var(--color-accent)` everywhere. When the accent color changes, one value changes and every element using it updates instantly, without reloading any component. This is the mechanism behind live theme switching. The second layer contains base components: Button, Input, Dialog, Card, and similar primitives built from shadcn/ui and styled with the token system. The third layer contains OS-specific components: `WindowChrome`, `AppShell`, and `NotificationToast`, which implement patterns specific to this desktop environment.

os-sdk. A shared Rust crate provides system integration to every first-party application. It bundles a typed Graph Daemon client (a Rust struct that opens the daemon's Unix socket and sends Cypher queries), an Event Bus client, MCP server boilerplate (the scaffolding to register tools and handle invocations), and the TOML-based config system with automatic schema validation. A new first-party application adds Tauri, ui-kit, and os-sdk and immediately has graph access, event emission, an MCP server,

config handling, and a consistent UI without writing any infrastructure code. The standard application layout reflects this:

```
app-files/
  src-tauri/
    Cargo.toml # includes os-sdk
    src/
      main.rs # Tauri setup
      commands.rs # Tauri command handlers
      mcp.rs # this app's MCP server
  src/
    main.tsx
    components/ # app-specific components (uses ui-kit)
    pages/
```

Listing 5 Standard first-party app structure

6.4 Spotlight

Spotlight is the system-wide launcher and command palette: the primary keyboard-driven entry point to the entire desktop, triggered by the Super key by default and accessible from any context as a centered overlay. It handles app launching, file and web search, inline calculation, unit conversion, timezone lookup, and contextual actions derived from the current selection or active application. It is not a file manager; deep folder navigation and bulk operations belong in the File Manager. Spotlight surfaces the result the user wants immediately or hands off to the right application.

When the AI layer is active, Spotlight gains an AI mode toggled by Tab. Natural language queries are interpreted by the AI layer, resolved against the Knowledge Graph and available MCP actions, and returned as structured results in the same interface. Asking ”show me what I worked on this week” is a Spotlight query, not a file search.

Third-party modules can register as spotlight.provider to add result types or as spotlight.action to add commands, as described in Chapter 7.4.

6.5 Notification System

Desktop notifications on Linux follow the *freedesktop.org Desktop Notifications Specification* [33], a D-Bus protocol that any application can use via libnotify [39] without knowing anything about the specific notification implementation on the system. An application calls notify_send() or the equivalent library function; the notification daemon receives it over D-Bus, decides how to present it, and handles persistence, grouping, and dismissal.

This project ships a first-party notification daemon written in Rust. Its architecture follows the pattern established by *swaync* [67] (SwayNotificationCenter), which demonstrated that the daemon logic and the visual presentation can be cleanly separated: the daemon handles the D-Bus interface, queuing, persistence, and routing policy, while the visual layer is a separate process that connects to the daemon. Rather

than adopting swaync's GTK-based frontend, this project replaces the visual layer with Tauri components that integrate with the design language and the Event Bus.

The notification system has two visual components. The *Toast Renderer* is a layer-shell surface anchored to the top-right corner of each monitor; it displays transient notifications for a configurable duration and dismisses them automatically. The *Notification Center* is a persistent panel surface that shows the full notification history, grouped by application, with actions and dismissal controls. Both are Tauri windows that receive notification data from the daemon over a Unix socket.

Every notification that passes through the daemon is written to the Knowledge Graph as a Notification node with timestamp, source application, summary, and whether it was interacted with. This gives the AI layer visibility into what the user was interrupted by and when, and enables queries like "what notifications did I miss while in the meeting?" without any changes to the applications that sent them.

6.6 Screen Lock

Screen locking on Wayland uses the ext-session-lock-v1 [88] protocol, a standard Wayland extension that allows a client to take exclusive control of all outputs and receive input until it explicitly unlocks. Once the lock surface is committed, the compositor freezes all other clients' rendering and routes all keyboard and pointer input to the lock client only. No other application can render to the screen or receive input events; this is enforced by the compositor, not by the lock client.

The lock screen is a first-party Tauri application that implements this protocol. On activation it acquires the session lock, renders its surface on every connected monitor, and communicates with PAM via a small Rust backend for credential verification. The visual design is part of the standard theme system and follows the same token architecture as the rest of the shell.

Lock activation is triggered by three paths: an explicit keyboard shortcut, the power menu, and idle timeout. Idle detection uses the ext-idle-notify-v1 Wayland protocol, which allows a client to register a timeout and receive a notification from the compositor when no input has been received for that duration. The idle-notify client is a minimal system daemon that triggers the lock screen on timeout and optionally suspends the system after a second, longer timeout.

The lock screen has no access to the graph, the event bus, or any user data beyond what the compositor provides (monitor layout and scale factors). It authenticates via PAM only. A failed unlock attempt is logged to the Audit Log as a security event; the lock screen itself does not write to the Knowledge Graph.

6.7 Multi-Monitor and HiDPI

Wayland treats multi-monitor and HiDPI as first-class concepts: each output carries its own scale factor, every Wayland-native application queries its output properties and renders at the correct scale automatically, and WebKitGTK respects the Wayland scale factor transparently. This means Tailwind layouts render identically on 4K and 1080p without extra work in the application.

The taskbar runs as a separate layer-shell surface per monitor, each correctly sized and scaled for its output. Monitor configuration, covering resolution, refresh rate, scale factor, position, and rotation per display, is stored in ~/.config/display/config.toml and applied at compositor startup. When a new monitor is connected, the compositor checks for a saved configuration and applies it immediately if found; for an unknown monitor it applies sensible defaults and surfaces a notification offering to open the display settings.

Fractional scaling at 1.5x (common on 1440p displays) uses the wp-fractional-scale-v1 Wayland protocol. Applications that implement the protocol render crisply at fractional scales; those that do not are rendered at the nearest integer scale, which results in slightly soft output but is acceptable and documented. XWayland has no per-monitor DPI concept and uses a single global DPI matched to the primary monitor; X11 applications do not re-scale when moved between monitors, which is a structural X11 limitation with no compositor-level fix.

6.8 Design Language and Theming

The specific visual design language, exact color palettes, and motion design are finalized in a separate design phase. This section documents the technical theming architecture: how themes are structured, how they propagate, and how community themes work.

Token system. All themes are defined in TOML and compiled to platform-specific outputs by the build system. The TOML file is the single source of truth; no theme value exists anywhere else. From one tokens.toml, the build generates:

- CSS custom properties for the Tauri shell and all first-party applications, applied via a root stylesheet loaded at startup.
- A GTK4 CSS theme, applied via gtk-theme-name in ~/.config/gtk-4.0/settings.ini.
- A Wine .msstyles file. Wine [19] is the compatibility layer that runs Windows applications on Linux; it has a visual theme system based on Microsoft's .msstyles format, the same format used for Windows XP visual styles. Generating one from tokens means Windows applications running under Wine use the same accent color and surface colors as the rest of the desktop.
- Base dark and light palettes for GTK3 and Qt.

Changing the accent color in the TOML file propagates to every output on the next build; in a live switch (see below), the CSS custom property update is applied without a rebuild.

Built-in themes. Four themes ship with the system. Dark and Light are standard high-contrast variants. High Contrast targets *WCAG AAA* [86] accessibility compliance, which requires a minimum contrast ratio of 7:1 between text and its background. WCAG (Web Content Accessibility Guidelines) is the international standard for digital accessibility maintained by the W3C; AAA is the highest of three conformance levels. The High Contrast theme uses no transparency effects, no gradients, and conveys no

information through color alone; it is generated from the same token system as the others and has both a dark and light variant.

Panda is the default and the most distinctive: a deliberate split-surface design where different UI regions are intentionally dark or light based on their function, creating a high-contrast frame around bright content. The exact palette values for all four themes are determined in the design phase; the token structure is fixed.

GTK and Qt coverage. GTK4 receives a full custom theme generated from tokens. GTK3 and Qt receive fixed base dark and light themes only; the split-surface Panda design requires component-level CSS control that neither framework exposes cleanly, so they fall back to whichever base variant is closer to the active theme.

Store themes. Community themes are distributed through the Store. A theme package declares which layers it covers in its manifest and can inherit from any built-in theme, overriding only the values it changes. The Store displays coverage before installation so users know which layers remain at the base theme. Two authoring formats are supported: token overrides for authors who want to change colors and radius values without touching CSS, and full compiled CSS for themes that need structural changes beyond what the token system exposes.

Live switching. Figure 9 shows the theme inheritance chain. When the user applies a theme, the Settings daemon loads the package, merges its token overrides with the base theme, and broadcasts updated CSS custom property values to all running Tauri processes via a Unix socket message. Each Tauri WebView has a JavaScript bridge that receives the update and writes the new values to the document's :root CSS rule; all elements that reference those properties via var() update immediately without a component reload or page refresh. This is what makes theme switching feel instantaneous for the Tauri layer. The GTK4 theme is regenerated and re-applied (which requires relaunching affected GTK4 applications to pick up the change), and Wine prefix themes are updated if applicable. Token-based switches complete in under a second; GTK4 regeneration may take one to two seconds on slower hardware.

7 Apps, Modules, and the Store

7.1 App Ecosystem

The application layer divides cleanly into two categories: first-party applications built as part of this project, and the broader ecosystem of third-party Linux software. Both operate under the same permission system and benefit from the same infrastructure, but they differ in how deeply they integrate with the platform.

First-party applications. All core system applications are built with the standard stack: Tauri, ui-kit, and os-sdk. Each one follows the shared design language, implements

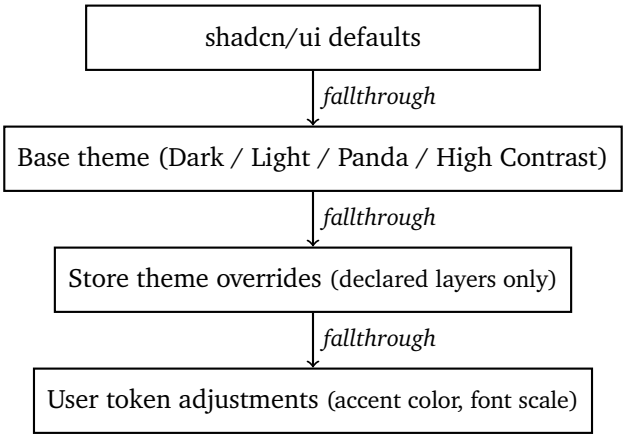

Figure 9 Theme inheritance chain. Each layer overrides only what it declares; everything else falls through to the layer below. A Store theme that covers only the shell and GTK4 leaves Wine and Qt at the base theme.

an MCP server for AI integration, emits structured events to the Event Bus, and uses the shared TOML configuration system. Table 12 lists the planned core applications.

Table 12 First-party system applications

Application	Notes
File Manager	Graph-aware features: recent files by project, related files sidebar
Terminal	MCP server exposing command history and session output
Settings	System-wide configuration: theming, permissions, AI level, accounts
Store	App and module discovery, installation, and updates
Text Editor	Basic editor primarily for configuration files
System Monitor	Process list, resource usage, and Anomaly Detector alerts
Wine Manager	Prefix management, Winetricks access, per-prefix version selection

Third-party integration layers. Third-party applications are not rewritten. Integration happens in four layers of increasing depth, each building on the one below.

At the baseline, every application on the system receives passive tracking via eBPF regardless of whether it does anything to support this platform: file access, network connections, and process lifecycle are recorded automatically. No changes to the application are required.

For applications that produce useful log output, a wrapper captures stdout and stderr and passes them through asynchronous LLM extraction to produce graph entities. These wrappers are shipped alongside packages in the package manager where practical, and the extraction runs in the background without blocking anything.

For popular applications, dedicated MCP server wrappers expose structured interfaces to the AI layer. These are maintained as separate packages and where possible contributed back to the community as independently useful tools.

Finally, applications that explicitly support the platform APIs emit structured typed events directly to the Event Bus and implement native MCP servers. This is the long-term goal for major open-source applications but is not a precondition for any of the layers above it.

Shared standards. Applications that choose to integrate with the platform work with a common set of conventions: TOML configuration files under ~/.config/<app-id>/, CSS custom properties from the token system for applications that render via WebView, the standard notification daemon via the Freedesktop D-Bus interface or the Event Bus for richer notifications, and a published MCP server interface specification.

7.2 Notification System

The Notification Daemon is the single entry point for all notifications from all sources. Figure 10 shows the pipeline.

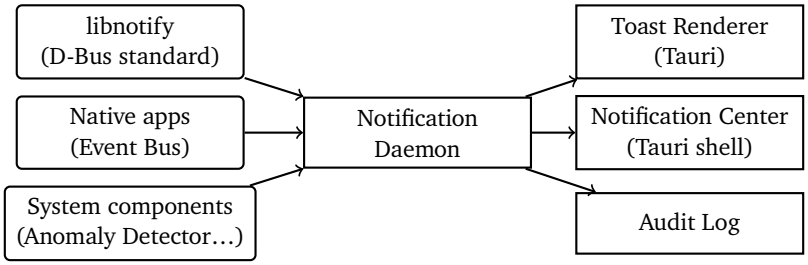

Figure 10 Notification pipeline. All sources converge on the Notification Daemon, which normalizes, groups, and applies priority logic before forwarding to the toast renderer and notification center. Every notification is logged in the Audit Log by sender and timestamp, never by content.

The daemon receives from two sources: the standard D-Bus interface at org.freedesktop.Notifications, which means every third-party application using libnotify or notify-send works without changes; and the Event Bus, through which native system components send richer notifications with priority metadata and graph context. Both are normalized to a common internal schema before any further processing.

Critical priority is reserved for system-level alerts only: Anomaly Detector warnings, component compatibility failures, and security events. Applications cannot self-assign Critical. Notifications from the same application are grouped in the Notification Center; as a toast, only the most recent notification from an application is shown, with a counter if a previous one from the same application is still visible. Do Not Disturb has three modes (off, on, and scheduled by time of day) and is togglable from the taskbar quick panel.

Table 13 Notification priority levels

Level	Behavior	Do Not Disturb
Critical	Persistent toast, no auto-close	Always shown, never silenced
High	Toast auto-closes after approximately 8 seconds	Delayed, not blocked
Normal	Toast auto-closes after approximately 4 seconds	Silenced
Low	No toast; goes silently to Notification Center only	Silenced

7.3 Permission System

Every application on the system, whether a first-party app, a Flatpak, or an RPM package, operates within a permission profile that defines what it can access. The system is unified: one file format, one enforcement layer, one place for the user to review and adjust everything.

Permission file format. Each application has a TOML permission file at ~/.config/permissions/<app-id>.toml that serves as the sole source of truth for what that application is allowed to do. The file is human-readable and user-editable; the Settings app provides a UI for those who prefer not to edit it directly. Permissions are enforced via Linux namespaces, AppArmor, and seccomp, described in Section 8.4.

```toml
# ~/.config/permissions/org.mozilla.firefox.toml

[filesystem]
allow = ["~/Downloads", "~/Documents", "~/.mozilla"]

[network]
allow = ["full"]

[devices]
allow = ["gpu"]

[graph]
# no entry = no access, never prompted
```

Listing 6 Example permission file

Graph access is never inferred or learned. An application with no [graph] section has no graph access, with no exceptions and no automatic prompting. Graph permissions must be explicitly declared.

Store-supplied profiles. The Store ships a curated permission profile alongside known applications. Common software such as Firefox, LibreOffice, and VLC has community-maintained profiles that are installed automatically and shown to the user at install time for review. Community members can submit profiles for applications not yet

covered; submitted profiles go through Store review before publication, since an overly permissive profile is a meaningful risk.

Flatpak adapter. Flatpak applications carry their own permission declarations. When a Flatpak is installed, the system automatically generates a native permission file by mapping Flatpak permissions to the native format: `filesystem=home` becomes `filesystem.allow = ["~/"]`, and so on. The generated file is shown to the user at install time and can be tightened before confirming. The Flatpak manifest is the floor; the system never grants more than what the Flatpak declared.

First-run learning mode. If no Store profile exists and the application is not a Flatpak, the first launch runs in learning mode: all accesses are permitted and logged, nothing is blocked. The learning window closes automatically ten minutes after first launch, or when the session ends, whichever comes first, the user receives a summary of what the application accessed, with options to review and confirm, edit first, or start with all permissions blocked. From the second launch onward, the confirmed profile is enforced. Learning mode applies to filesystem, network, and device access only; graph access is always denied until explicitly granted and cannot be learned. During the learning window, at most five new permission grants are recorded per minute; exceeding this rate suspends learning immediately and generates an anomaly alert. This limits the damage from a compromised application that attempts to accumulate permissions in bulk during its first session.

Runtime behavior. When an application attempts an access outside its profile, the access is denied and the user receives a notification offering to extend the profile. If the user allows, the profile is updated and the application retries the action; if the user denies, the block stands. Repeated denials for the same path are collapsed: after three denials for the same access, the system asks once whether to always block it rather than continuing to prompt.

Ephemeral app mode. Any application can be launched in ephemeral mode, either from the right-click context menu in the launcher or via a command-line flag. Ephemeral mode combines several existing sandbox mechanisms into a single preset with no configuration required.

In ephemeral mode, a private Mount Namespace is created with no write access to the home directory; the application sees a temporary overlay that is discarded on exit. Graph access is entirely disabled regardless of the application's normal permission profile. Network activity is permitted if declared in the application manifest, but is not logged to the Audit Log. On exit, the temporary overlay is deleted, no entries are written to the Knowledge Graph, and no permission changes persist.

The intended use case is opening untrusted or one-off content: a PDF from an unknown sender, a binary downloaded for a single use, or any application the user wants to run without leaving a trace. Ephemeral mode is not a replacement for the full profile isolation that separate Linux users provide; it does not protect against a compromised application exploiting a kernel vulnerability. What it provides is a

low-friction way to reduce the footprint of routine untrusted content without requiring a separate session or a virtual machine.

7.4 Module System

The system is extensible at multiple layers through a unified module mechanism. All module types use the same installation channel (the Store) and the same manifest format, but they operate as independent subsystems with separate APIs. A single Store package can bundle multiple module types together.

Module categories. Table 14 lists the available extension points. Theme modules are a special case: they consist of CSS and token overrides only, contain no executable code, and require no sandbox. All other categories run sandboxed with declared permissions.

Table 14 Module categories and extension points

Category	Extension point	Example
theme	Design language layer	Community color schemes
spotlight.provider	Spotlight search results	Web search, stock prices
spotlight.action	Spotlight command actions	Create task, send message
shell.topbar	Topbar element slot	Custom status widget
shell.widget	Desktop widget area	Clock, weather, system stats
mcp.server	AI MCP integration	External service connector
keybinding.profile	Global input manager	Alternative layout profiles

Manifest format. Every module package has a single manifest that declares what it provides, its dependencies, conflicts, and permissions. The [permissions] block uses the same schema as application permission files; at install time the Module Runtime generates the permission file from the manifest and the user can inspect and adjust it in Settings like any other application.

```
[package]
id = "com.example.ai-spotlight"
name = "AI␣Spotlight␣Provider"
version = "1.0.0"

[provides]
spotlight.provider = true

[dependencies]
shell = ">=1.2"
modules = ["com.example.ai-core"]

[permissions]
network.allow = ["api.openai.com"]
graph.allow = ["read"]
```

Listing 7 Example module manifest

Modules cannot declare permissions beyond what the manifest specifies, and manifests are reviewed during Store submission. Unlike applications, modules cannot expand their permissions at runtime through a learning mode; their profile is set at install time and any undeclared access returns `EACCES`.

Module Runtime. A dedicated Module Runtime daemon manages lifecycle, sandboxing, and crash recovery for all active modules. Each module runs in its own isolated WebView context, separate from the shell's WebView. A module crash does not affect the shell or other modules.

The runtime uses exponential backoff per module instance for crash recovery, shown in Table 15. The 60-second window resets on a clean run, so a module that runs for several minutes and then crashes once gets an immediate restart.

Table 15 Module crash recovery policy

Crash count (within 60 s)	Behavior
1st	Immediate restart
2nd	Restart after 5 seconds
3rd	Restart after 30 seconds
4th or more	Marked as failed; no further automatic retries

When a module is marked failed, Settings shows a subtle error indicator with the last crash log. The user can trigger a manual retry at any time. There is no notification spam.

Module updates are downloaded in the background and applied at different points depending on category: theme updates apply immediately as a token and CSS swap; Spotlight modules update on the next Spotlight open; shell widgets and topbar modules update on the next session start by default, with a manual "Restart now" option available in Settings; MCP server modules update on the next session start to avoid interrupting active AI interactions.

Shell API. Modules interact with the system through a restricted TypeScript SDK. No direct access to the Smithay compositor or Graph Daemon is available; what is not in the SDK is not accessible.

```
import { shell } from "@os/module-sdk"

// Window management
const windows = await shell.getWindows()
const active = await shell.getActiveWindow()

// Graph (requires graph = "read" in manifest)
const recent = await shell.graph.query(
  "MATCH␣(f:File)␣RETURN␣f␣LIMIT␣5"
```

```
)

// Spotlight provider registration
shell.spotlight.register({
  id: "com.example.ai-spotlight",
  placeholder: "Ask␣AI...",
  onQuery: async (q) => [ /* result objects */ ]
})
```

Listing 8 Module SDK surface (TypeScript)

Global Input Manager. Keybindings and gestures are configured in a single place in Settings, not per application or per module. Modules register named actions with default bindings; the user overrides them centrally. The module never receives the binding string directly; the Input Manager calls the registered action handler when the binding fires. This means a user can rebind everything from one place and no module can silently capture input outside its declared actions.

Developer experience. A CLI scaffolds a complete module project with one command: `npx create-os-module <name>`, producing a Vite, TypeScript, Tailwind, and module-sdk setup with a manifest template. In dev mode, the module runs unsigned in a dedicated dev sandbox, all declared permissions are granted without Store review, WebView DevTools are accessible, and hot-reload via Vite is active. Dev mode is only available on developer builds; production builds reject unsigned modules.

For distributing a finished module outside the Store, a developer generates a self-signed key pair with the CLI, signs the module, and distributes it as an `.osmod` file. Installation uses TOFU: the first install prompts the user to trust the key fingerprint; subsequent updates from the same key are silent. Sideloaded modules receive no expanded permissions and are marked "Not from Store" in Settings.

7.5 Store

The Store is the primary distribution channel for applications, themes, modules, and profile packages. It is designed to be financially sustainable for the project without becoming a gatekeeper for the ecosystem.

Publishing. Publishing an application to the Store is free and remains free indefinitely. There is no annual developer fee. Every upload runs automated technical checks: valid permission declarations, correct package signing, sandbox policy compatibility, and resolvable declared dependencies. These checks are automated; no manual review is required for packages that pass them. A failed check returns a specific error. A manual Security Review is available optionally and is the basis for the Security Audit Badge described below.

Revenue model. Supported pricing models include free, one-time purchase, subscription, in-app purchases, pay-what-you-want, and freemium. Developers can also direct

users to external payment links; the Store payment infrastructure is a convenience, not a monopoly.

Commission is calculated on total annual revenue across all applications in a developer account, using the tiered structure in Table 16. When a threshold is crossed mid-year, the new rate applies to all subsequent transactions in that year, not retroactively.

Table 16 Store commission tiers

Annual revenue (account total)	Commission
Up to 10,000€	0%
10,000€ to 50,000€	5%
50,000€ to 500,000€	15%
500,000€ to 5,000,000€	20%
Over 5,000,000€	25%

User-facing features. Purchases are tied to the user account, not the device. Trials are declared in the app manifest and enforced by the Store; no credit card is required to start a trial, and when it ends the user receives a single notification with a purchase option and no automatic charges. Family Sharing is opt-in per application: developers can allow up to six to eight family members to use an application under one purchase. Every application's Store page shows a full price history including every price change with its date; this is automatic and cannot be disabled by the developer. All active subscriptions across all applications are visible and cancellable in one place in Settings.

Developer dashboard. Developers see install counts, active user numbers, revenue and payout history, ratings and written reviews, crash reports from users who opted into crash reporting at the system level, and version adoption numbers. All data is aggregated and anonymized; the developer sees counts, never identities. No third-party analytics are required or supported.

Security Audit Badge. Applications can display a Security Audit badge on their Store page by uploading an independent security audit report. The Store verifies that the report originates from the stated auditing organization and links it from the application page. Users can read the full report and judge the auditing organization's credibility themselves. The badge means an audit exists and is accessible, not that the Store has endorsed the application's security.

Long-term support. Developers can declare a Long Term Support commitment in the application manifest by setting a support end date. The Store displays this date publicly on the application page. It is a voluntary public commitment with no automated enforcement, but the public visibility creates accountability. LTS applications are filterable in Store search for users who prioritize stability.

Profile package distribution. Signed environment profile packages (.lenv files, described in Section 8) are distributed through the Store using the same infrastructure as applications. Profile packages are free to publish and carry no commission.

8 Platform, Profiles, and Security

8.1 Account System

The Account System is the single place where the user manages all online accounts. Set up once, available everywhere, with full visibility into which application accesses what. The design principle is straightforward: no application ever directly handles OAuth flows, stores tokens, or manages credentials. All account logic lives in a dedicated *Account Daemon*.

OAuth 2.0 [41] is the standard authorization framework used by cloud services to grant third-party applications access to user data without sharing the user's password. The flow works by redirecting the user to the service's login page, where they authenticate and approve a specific set of *scopes* (declared permissions such as "read email" or "write calendar"), after which the service issues a short-lived *access token* and a long-lived *refresh token*. The access token is sent with each API request; the refresh token is used to obtain new access tokens when the current one expires. An application that holds both tokens has persistent access to the user's account; protecting them is therefore critical.

Architecture. Applications declare what they need; the daemon handles the rest. When an application needs access to a service, it sends a request to the Account Daemon, which checks the application's permission profile, retrieves or derives the appropriate token, logs the access in the Audit Log, and returns what the application needs. The application then communicates with the provider's API directly using that token. The daemon is the sole keeper of all credentials; tokens never live in application memory longer than necessary.

Service-level permissions. Applications request access at the service level, not the OAuth scope level. The user sees human-readable service names and never raw OAuth scope strings. When an application requests account access for the first time, the user sees exactly which service is being requested, grants or denies it, and the decision is stored in the application's unified permission profile as described in Section 7.3. Account permissions are part of the same system as filesystem, network, and graph permissions.

```
Google Account (personal@example.com)
  Mail -> allowed for: Thunderbird
  Calendar -> allowed for: Calendar App, AI daemon (read-only)
  Drive -> allowed for: File Manager
  Contacts -> not granted to anyone
```

Listing 9 Service-level permission view (Settings)

Multiple accounts and resolution order. When multiple accounts of the same type exist, the system resolves which to use via a three-tier model. The system default applies globally; an Isolated Desktop context can declare its own defaults, so that switching to the Work desktop automatically switches the active account context; and individual applications can be manually pointed at a specific account regardless of the desktop context. Resolution proceeds from most specific to least specific: per-app override, then desktop default, then system default.

```
# resolved in order: per-app override -> desktop default -> system default
[defaults]
google = "personal@example.com"

[desktop.work.defaults]
google = "work@example.com"

[app."org.mozilla.thunderbird"]
# always uses personal account regardless of active desktop
google = "personal@example.com"
```

Listing 10 Account resolution configuration

Token security. Credentials never leave the Account Daemon as-is. Two paths are used depending on provider support. For providers that support OAuth scope restriction, the daemon issues a short-lived derived token scoped only to what the application declared, with a default lifetime of five minutes. The full OAuth token and refresh token never leave the daemon at all. For providers that do not support downscoping, the full token is handed to the application but with two compensating controls: every token handoff is logged in the Audit Log, and tokens are actively revoked when the application process exits. All credentials at rest are stored in the Linux Kernel Keyring [11]: a kernel-managed storage area for cryptographic keys, tokens, and passwords that lives in kernel memory rather than userspace files, making it inaccessible to other processes and not visible in `/proc` or on disk. The daemon loads them into memory only when actively needed and discards them immediately after.

Offline behavior. Two failure modes are handled distinctly. When the provider rejects the token because the user revoked access or the account password changed, the daemon emits a single system notification directing the user to Settings and does not repeat it; applications receive a clear `AccountAuthRequired` error code and surface a gentle in-app hint. When the network itself is unavailable, the daemon serves cached data from the last successful sync, clearly marked with the timestamp of the last update. A calendar application shows last-known events labeled with their age rather than an empty state. When connectivity is restored, the daemon refreshes silently in the background and notifies applications of updated data via the Event Bus.

GNOME Online Accounts bridge. *GNOME Online Accounts* [37] (GOA) is a GNOME framework that centralizes cloud service credentials (Google, Microsoft, Nextcloud, and others) and exposes them to applications via a D-Bus interface. Applications built

for GNOME that use GOA expect a D-Bus interface at org.gnome.OnlineAccounts. The system provides a GOA-compatible bridge that implements this interface exactly but sources all data from the Account Daemon internally. Evolution, GNOME Calendar, and other GOA-dependent applications work without modification. Internally every request goes through the Account Daemon, through the permission system, and through the Audit Log. The bridge translates; it does not add logic.

8.2 Profile System

A profile is a complete, isolated user environment backed by a separate Linux system user. Each profile has its own home directory, its own Knowledge Graph, its own account configuration, and its own set of running system daemons. Isolation is enforced at the kernel level, not by convention.

The profile system replaces the concept of isolated desktops within a session. Rather than trying to isolate individual desktops within a single session, the entire session is the isolation boundary. Switching profiles is a session switch, not a window toggle.

Personal profiles. The primary profile, created at system setup, cannot be deleted. It has no restrictions: full access to all installed applications, all accounts, and all system features. This is the default context the user logs into.

Work and custom profiles. A user-created or organization-deployed profile for a specific context such as work, a project, or a secondary identity. Backed by a separate Linux user with its own encrypted home image via systemd-homed. Virtual desktops, accounts, Knowledge Graph, and application configurations are all independent from the Personal profile. Multiple custom profiles can coexist. Each is a full Linux user; switching to one is a standard session switch with saved and restored window state.

Locked profiles. A profile deployed by an institution for controlled sessions: exams, regulated work environments. Backed by a temporary Linux user created at session start and deleted when the session ends. The user cannot switch to another profile while a Locked session is active; this is enforced at the compositor level, not by UI alone. After the session ends, the Linux user and all associated data are removed. No data from the Locked session survives into the Personal profile.

Guest mode. A lightweight temporary context for short-term use by another person. Unlike other profile types, Guest does not create a second Linux user or a second compositor session. It runs as a sandboxed environment on top of the current session: a separate namespace, an empty home directory, no access to the host user's data. The host session is suspended in the background and resumed instantly when Guest ends. All Guest data is discarded on exit.

Technical foundation. Each non-Guest profile is a Linux system user. Profile creation is useradd plus systemd-homed home image setup; profile switching is a standard Linux user session switch; profile deletion is userdel plus home image removal. Isolation is

kernel-enforced. systemd-homed provides per-profile LUKS-encrypted home images. When a profile is not active, its home image is unmounted and its encryption key is discarded from memory. Each profile runs its own instance of all system daemons: Graph Daemon, Account Daemon, Event Bus. There is no shared daemon state between profiles.

Data transfer between profiles. By default profiles are fully sealed from each other. Controlled data transfer is configured per profile pair in Settings, with independent directional control over clipboard, drag-and-drop, and a shared folder:

```
[profile.work.transfer]
clipboard = "in-only" # "off" | "in-only" | "out-only" | "both"
drag_drop = "in-only"
shared_folder = "~/Transfer/Work"
shared_folder_permission = "read-write" # or "read-only" | "off"
```

Listing 11 Cross-profile transfer configuration

A dedicated *Transfer Daemon* is the only process with simultaneous access to both profile namespaces. All cross-profile data flow goes through it; nothing bypasses it. Whenever data flows out of a profile, a non-dismissable toast appears for three seconds identifying the source profile, destination, and file name, with an Undo option. The transfer proceeds but can be cancelled within the window. All transfers are recorded in the Audit Log with source, destination, transfer type, and data description.

Locked profiles have transfer hardcoded to off in all directions. This cannot be overridden by the user or by an organization's configuration package; it is a system-level invariant with no configuration surface.

Signed profile packages. Profiles can be deployed via a *Signed Profile Package*, a .lenv file created and signed by an organization. The user installs it themselves and retains full control over their device. The organization gets a tamper-evident, versioned configuration. This is the BYOD model: the organization controls their context, the user controls their device.

```
[environment]
id = "com.acme.work-profile"
name = "Acme␣Work"
version = "2.1.0"
publisher = "Acme␣GmbH"
type = "persistent" # or "locked" for exam/controlled sessions

[transfer]
clipboard = "in-only"
drag_drop = "in-only"
shared_folder = "off"

[network]
require_vpn = "acme-vpn"
allow_outbound = ["*.acme.com", "*.office365.com"]
block_outbound = ["*"]

```

```
[apps]
required = ["com.acme.timetracker", "org.mozilla.firefox"]
blocked = ["com.valve.steam"]

[graph]
ai_access = "off"
export = "off"
```

Listing 12 Signed profile package format

The package is signed by the organization's private key. Key distribution uses TOFU by default: on first install the user sees the publisher name and key fingerprint and confirms trust. For managed onboarding, IT can send an enrollment link so that the organization's key is imported before any package arrives, eliminating the TOFU prompt entirely.

Before installation, a full summary of every policy and restriction is shown. Nothing is hidden. There are certain things an organization cannot do through a signed package, enforced technically rather than by policy: the organization cannot read the user's Personal profile or its Knowledge Graph, cannot access any data outside the profile's Linux user boundary, cannot monitor activity in other profiles, cannot remotely wipe or modify the device, and cannot prevent the user from uninstalling the package. If `notify_on_uninstall` was declared and accepted at install time, the organization receives a notification when the package is removed, but the removal proceeds regardless.

8.3 Gaming and Windows Compatibility

Gaming and Windows application compatibility are first-class features, not afterthoughts. The required components are pre-installed and pre-configured; no manual setup is required.

Wine and Proton. *Wine* provides compatibility for Windows applications on Linux. *Proton* [82] is Valve's fork of Wine, optimized for gaming and maintained with extensive patches from Valve's Linux gaming team; for gaming specifically it is substantially better than vanilla Wine. Both are pre-installed alongside *DXVK* [26], which translates Direct3D 9, 10, and 11 API calls to Vulkan. Direct3D is the Windows graphics API; most games targeting Windows use it rather than Vulkan directly. By translating at the API boundary, DXVK allows these games to run on Linux via the Vulkan driver. *vkd3d-proton* [83] is Valve's fork of the Wine project's vkd3d library; it translates Direct3D 12 (the modern Windows GPU API) to Vulkan, with extensive optimizations for gaming workloads that upstream vkd3d does not include. Winetricks is included for installing Windows runtime components into Wine prefixes.

Steam is available in the Store and works out of the box with Proton enabled. Steam manages its own Proton version per game; the system does not interfere with Steam's internal Proton management.

Non-Steam Windows games and applications use Wine directly, managed through the first-party *Wine Manager* application. The underlying components are shared

with Steam's installation rather than duplicated. Wine Manager is a thin UI over the existing tooling providing prefix creation and deletion, per-prefix Winetricks access, launch shortcuts, and per-prefix Proton and Wine version selection.

DXVK and vkd3d-proton require Vulkan. Mesa provides Vulkan support for AMD and Intel GPUs by default. Nvidia requires the proprietary driver for Vulkan support; the Store provides explicit guidance for Nvidia users during setup.

Wine theming. By default Wine applications render with a Windows Classic or Windows 7-era appearance that is visually inconsistent with a modern desktop. A first-party Wine theme is maintained as part of this project, generated from the same design token system used for GTK and Qt theming. It updates automatically when the system theme changes. Wine applications do not look native, but they look consistent with the rest of the system. The Wine theme is applied automatically to all Wine prefixes managed by the system; users who prefer the default Windows appearance can opt out per prefix.

Performance tooling. *Gamemode* [31] by Feral Interactive is pre-installed and enabled by default. It applies system-level performance optimizations when a game is running: the CPU governor switches to performance mode, the I/O scheduler is tuned, and kernel scheduler hints are applied. Proton handles the Gamemode activation call automatically; no manual configuration is required.

MangoHud [53] is pre-installed as an in-game overlay showing frames per second, GPU and CPU usage, temperatures, and frame timing. It is disabled by default and toggled via a keyboard shortcut.

Knowledge Graph integration. Gaming activity is tracked in the Knowledge Graph like any other activity: which games were played, when, and for how long; which Wine and Proton versions were used per game; and performance data from MangoHud when enabled, linked to sessions. This makes queries such as "how many hours did I play last week" or "which Proton version worked best for this game" available locally without relying on any external service.

8.4 Security and Privacy

Security is not a feature added on top of this system. It is a foundational design constraint that shapes every architectural decision.

Core principles. Five non-negotiables drive everything in this section. First, the Knowledge Graph, Audit Log, and AI context are local by default and never leave the machine without explicit user action; there is no opt-out dark pattern, the default is that nothing goes anywhere. Second, the system collects no usage data, crash reports, or any other information without explicit user consent. Third, the system works fully offline and without registration; cloud features are opt-in. Fourth, every component gets exactly the permissions it needs and nothing more, applying to filesystem access, network access, graph access, and syscalls alike. Fifth, security must not destroy usability: de-

faults are chosen to protect without requiring constant interaction, because a system so locked down that users disable its protections is less secure than a well-calibrated one.

A sixth principle follows from the others: data minimization is itself a security property. Data that does not exist cannot be stolen. The retention policy defined in Section 4.5 is not only a privacy decision but an active security measure. Every decision to extend retention windows is a deliberate expansion of attack surface and must be understood as such.

8.4.1 Threat Model

Security measures without a named adversary are hard to evaluate. This section states explicitly what this system is and is not designed to resist, so that the measures described in the sections that follow can be judged against a concrete baseline.

In scope. The primary adversary is a *local attacker with user-level privileges*: a malicious or compromised application running as the logged-in user that attempts to exceed its declared permissions, exfiltrate graph data, escalate privileges, or persist across sessions. The secondary adversary is an *opportunistic remote attacker* who exploits a network-reachable vulnerability in a running service or a browser-class application. The tertiary adversary is a *physical attacker* with brief unsupervised access to a powered-off or locked device. Managed-environment scenarios add a fourth adversary: a *malicious or over-reaching administrator* who attempts to access user data beyond the boundaries described in Section 8.5.

Out of scope. This system does not attempt to resist a *root-level attacker*: an adversary who already holds superuser privileges on the running system. Root can modify kernel modules, bypass all userspace controls, and read any unencrypted memory. Full Disk Encryption and TPM-sealed keys protect data at rest against root on a different boot, not against root in the current session. *State-level adversaries* with hardware implant capabilities, supply-chain compromise of the base distribution, or CPU microcode vulnerabilities are similarly out of scope; no single OS can address these without hardware-level attestation infrastructure outside the scope of this project.

Knowledge Graph as a high-value target. The Knowledge Graph introduces a risk that does not exist on a conventional Linux system: a structured, queryable, cross-application timeline of user activity. On a conventional system, an attacker who gains access to a user's files reads unindexed data spread across many directories. An attacker who gains access to the running Graph Daemon gains a pre-indexed, graph-traversable record of everything the user has done. This is structurally more valuable than a raw filesystem breach, and the security model of this system must account for it explicitly.

The measures described in Section 8.4.5 reduce the probability of such a breach. Section 8.4.5 addresses what happens when prevention fails.

8.4.2 Full Disk Encryption

Full Disk Encryption means that everything stored on disk is encrypted at rest. On Linux this is implemented via *LUKS* on top of the *dm-crypt* kernel module. The entire partition is encrypted and the OS decrypts it transparently at boot after credentials are provided.

FDE is enabled by default. A stolen or lost device is a realistic threat for any user; without FDE, physical access means full access to all data. The performance concern that historically argued against FDE no longer applies: all modern x86 and ARM processors have hardware AES acceleration that makes encryption effectively free at current workloads.

The primary unlock mechanism is a passphrase entered at boot. On systems with a *TPM* (Trusted Platform Module) [16], a dedicated security chip on the motherboard (or firmware-emulated since TPM 2.0) that provides hardware-backed cryptographic operations, secure key storage, and tamper-evident measurements of the boot process. The FDE key can be sealed to the TPM and released automatically if the system boots in a known-good state. This allows passwordless boot on trusted hardware while still protecting the drive if it is physically removed. If the boot configuration changes due to tampering, the TPM refuses to release the key and falls back to the passphrase. TPM-based unlock is opt-in; passphrase is always the fallback.

Hibernate requires additional care because the entire RAM contents are written to the swap partition. That swap partition must also be LUKS-encrypted, and key management must allow the system to resume without manual intervention. This is a known-solved problem on Linux but requires deliberate configuration during the installation process.

8.4.3 Secure Boot, Unified Kernel Images, and Measured Boot

Secure Boot [17] is a UEFI [80] firmware feature that verifies the cryptographic signature of every piece of code executed during the boot process. If any component is not signed by a trusted key, the firmware refuses to execute it. The practical effect is that an attacker who modifies the bootloader or kernel cannot boot the modified system without also possessing the signing key.

Both candidate base distributions support Secure Boot via a Microsoft-signed shim bootloader. Custom kernel modules built for this project must be signed with a Machine Owner Key enrolled in the firmware.

A traditional Linux boot chain consists of multiple separate files: the bootloader, the kernel image, and the initramfs. A *Unified Kernel Image* [18] packages all three into a single signed EFI binary, reducing the attack surface to a single component requiring verification. `systemd-boot` supports UKI natively and the broader Linux ecosystem is moving in this direction. This project targets UKI as the boot format.

Measured Boot goes further than Secure Boot: every boot component is cryptographically hashed and the hash is stored in the TPM's Platform Configuration Registers. This creates a tamper-evident record of exactly what ran during boot. If any component changes, even legitimately due to an update, the PCR values change. Locally, the TPM can be configured to release the FDE key only if the PCR values match an expected set, so that a modified bootloader causes the TPM to refuse key release. In

managed environments, a remote server can request a signed attestation report from the TPM, and a device that fails attestation can be denied network access.

Secure Boot combined with *Measured Boot* makes the Evil Maid Attack significantly harder. An Evil Maid Attack is the scenario where an attacker has brief physical access to a powered-off device and modifies the bootloader to capture the user's FDE passphrase on the next boot. Without the signing key, Secure Boot blocks the modified bootloader from executing entirely.

8.4.4 App Sandboxing

Every application runs in a restricted environment that limits what it can do even if fully compromised. Three mechanisms operate simultaneously and independently; bypassing one does not bypass the others.

Table 17 Sandboxing mechanism layers

Mechanism	Function
Linux Namespaces	Changes what the application can see
AppArmor	Blocks actions on resources the application can see
seccomp	Blocks specific syscalls regardless of resource visibility

Linux Namespaces give each application its own restricted view of the system before any permission check occurs. A Mount Namespace means the application sees only its own data directory, explicitly granted paths, and a minimal set of system paths; other applications' data directories do not exist from its perspective. A Network Namespace means applications without declared network access see no network interface at all; applications with declared network access see only a filtered virtual interface. A PID Namespace means the application can only see its own process tree.

AppArmor profiles are defined externally in the package and reviewed during Store submission; an application cannot define or modify its own profile. A text editor does not need `ptrace`, `mount`, or `kexec`; a seccomp profile that excludes these syscalls causes the kernel to kill the process immediately if a compromised application attempts them.

IPC is restricted to the Event Bus, the Graph Daemon, and explicitly declared inter-application communication channels. An application cannot open a socket to another application without that channel being declared and approved.

Clipboard hygiene. The clipboard is an underappreciated data-leak vector. On a conventional Wayland desktop, any application with a `wl_data_device` interface can read the current clipboard contents without restriction; a password manager and a text editor have equal access to whatever the user last copied. This system changes that.

Every clipboard entry carries a sensitivity label. Labels are assigned in two ways. Manual labeling is available from the clipboard history UI: the user can mark an entry as sensitive, causing it to be hidden from clipboard history and restricted from low-trust applications. Automatic labeling applies heuristics at copy time: content that

matches the pattern of an API key, an IBAN, a private key, a password-manager field, or a credit card number is labeled sensitive automatically without user interaction.

Applications declare a clipboard permission level in their manifest, analogous to filesystem or network permissions. An application without the clipboard.read.sensitive permission receives an empty string when it requests clipboard contents that carry a sensitive label. The application is not informed that content exists but was withheld; from its perspective the clipboard is empty. This prevents a compromised application from silently harvesting credentials that were copied from a password manager.

Clipboard entries labeled sensitive are excluded from clipboard history storage entirely and are not written to the Knowledge Graph. They exist only in the compositor's in-memory clipboard buffer and are cleared when the entry is superseded or when the screen locks.

8.4.5 Knowledge Graph Security

The Knowledge Graph is the most sensitive component in the system. It accumulates a detailed record of everything the user does and its access control must be correspondingly rigorous.

No application has direct access to the Kuzu database files. All graph access goes through the *Graph Daemon*, a privileged system process that is the sole reader and writer of the graph. Every other component communicates only with the daemon over a defined IPC interface, and the daemon checks permissions before executing any query.

Three axes of permission. Graph permissions are defined along three independent axes. The first axis is node types: which categories of data the caller can read, such as File, App, NetworkConnection, or Session. A calendar application may read Session nodes but not NetworkConnection nodes. The second axis is relations: which connections between nodes the caller can traverse. An application permitted to read File nodes may still be blocked from traversing the OPENED_BY relation that connects files to the applications that opened them. The third axis is instances: whether the caller can see only the nodes it generated itself or nodes generated by other applications as well.

```
[app."com.example.notes"]
allow_read = ["File.path", "File.last_modified", "Session.started_at"]
deny_read = ["File.content_snippet", "NetworkConnection.*"]
allow_relations = ["PART_OF_SESSION"]
instance_scope = "self" # only nodes this app generated
```

Listing 13 Graph permission entry

Two permission layers. Permissions are assigned in two layers. At install time, the package declares the maximum graph permissions it can ever receive; the user reviews this declaration as part of installation. At runtime, the application can request permissions within its declared maximum, with a user prompt explaining what is requested and why. A runtime request that exceeds the install-time declaration is rejected outright. Some fields within permitted node types are additionally restricted

regardless of node-level permissions: file content snippets, for example, require an explicit extra permission separate from general File node access.

Kuzu file protection. The Kuzu database files are protected by three independent layers. At the filesystem layer, the Graph Daemon runs as a dedicated system user graph-daemon and the database files are owned exclusively by this user with 600 permissions. A successful sandbox escape by an application process running as the logged-in user does not grant access to these files. At the AppArmor layer, a separate deny rule in every application's AppArmor profile prohibits access to the graph directory even if filesystem permissions were bypassed. At the encryption layer, the graph directory lives inside a LUKS-encrypted loopback image that the daemon mounts at startup using a key retrieved from the Kernel Keyring. When the daemon stops, cleanly or via crash, the image is unmounted and the key is discarded from memory. During any period when the Graph Daemon is not running, the Kuzu files are LUKS-encrypted ciphertext unreadable even to root.

Cypher injection prevention. The Graph Daemon never accepts query strings with embedded values. All queries must use named parameters, and the daemon runs a validator on every incoming query that detects embedded values before execution.

```
// Rejected - string interpolation
let query = format!("MATCH (f:File {{name: '{}'}}) RETURN f", user_input);

// Required - parameterized
daemon.query(
    "MATCH (f:File {name: $name}) RETURN f",
    params!{ "name" => user_input }
)
```

Listing 14 Parameterized graph queries

Applications without arbitrary_query permission in their manifest cannot send free-form query strings at all; they select from pre-approved query templates and supply only the parameters. Only explicitly trusted components such as the AI daemon and system daemons receive arbitrary_query permission, and even they go through the parameterization validator.

Capability tokens. Instead of looking up permissions in an ACL table on every query, applications receive a cryptographically signed capability token at startup. The token encodes the application's allowed scopes directly, and the daemon verifies the *HMAC* (Hash-based Message Authentication Code) signature on each query without a database lookup: the verifier recomputes the HMAC and checks it matches without needing to contact any authority. This is faster at high query rates and more robust against a corrupted ACL table. The AI daemon can derive restricted sub-tokens for specific query scopes without involving the daemon. Token revocation when permissions change mid-session is handled via a small in-memory revocation list that is cleared on daemon restart.

Token issuance and the Policy Store. Token issuance is separate from token verification and requires careful design: if the component that signs tokens can be influenced to grant excessive scopes, the entire permission model is undermined.

Tokens are signed by the Account Daemon at application startup. The permitted scopes for each application are not decided by the Account Daemon itself; instead, the daemon reads them from a *Policy Store*, a static TOML file owned by root with 644 permissions (world-readable, root-writeable only). The Policy Store maps application IDs to their maximum permitted graph scopes:

```
[app."com.example.notes"]
graph_read = ["File.path", "File.last_modified", "Session.started_at"]
graph_write = []
instance_scope = "self"
arbitrary_query = false
```

Listing 15 Policy Store entry (read-only by Account Daemon)

The Account Daemon can only issue a token whose scopes are a subset of what the Policy Store declares for that application ID. It cannot grant scopes that are absent from the Policy Store entry. A compromised Account Daemon can sign fraudulent tokens, but only within the ceilings the Policy Store enforces — because the Graph Daemon independently reads the Policy Store and rejects any token whose scopes exceed the registered maximum for that application. Scope expansion requires a root-level file write, which is an observable, auditable action. New applications receive a restrictive default scope at install time; privilege elevation is explicit and logged.

Graph Daemon hardening. The daemon runs with the minimum privileges needed for its function, enforced by a systemd service unit with PrivateNetwork=true, MemoryDenyWriteExecute=true, PrivateTmp=true, and a tightly scoped SystemCallFilter. PrivateNetwork=true prevents a compromised daemon from making outbound connections to exfiltrate data. A systemd watchdog monitors the daemon; an unresponsive or crashed daemon triggers a clean shutdown sequence that unmounts the LUKS image and discards the key before restarting.

Clients verify that they are speaking to the legitimate Graph Daemon before sending queries. At startup, the daemon signs a challenge with a private key that only it holds; clients verify the signature against the public key distributed with the system package. A connection that fails verification is dropped, preventing a trojanized replacement daemon from intercepting graph traffic.

Read/write process split. A single daemon process that both writes graph data and answers queries is a larger attack surface than necessary. The Graph Daemon is therefore split into two cooperating processes: graph-daemon-write accepts events from the Event Bus and writes to Kuzu, and graph-daemon-read answers queries from applications and daemons.

graph-daemon-read opens the Kuzu database in read-only mode. A compromised read process cannot modify graph data. graph-daemon-write does not expose a query interface to external callers; a compromised write process cannot be queried for data it has written. The two processes communicate only via a restricted internal channel

to coordinate schema changes and to propagate index updates. Both remain subject to the full systemd sandboxing described above.

This split does not eliminate the risk of a compromised read daemon leaking data it has legitimate access to. It does prevent an attacker who gains code execution in one path from trivially pivoting to the other.

Compromise scenario and damage control. The measures above reduce the probability of a Graph Daemon compromise. They do not make it impossible. An attacker who successfully executes arbitrary code inside `graph-daemon-read` can issue arbitrary Cypher queries against the open database handle and receive responses over the Unix socket, regardless of the capability token the calling client presented. This is the honest consequence of any architecture where a process must be running and reachable to serve data: the running process is the trust boundary.

Several controls bound what an attacker can do from this position. `PrivateNetwork=true` means the data cannot be sent over the network from the daemon process itself; exfiltration requires additional steps through another compromised component. The read-only Kuzu handle prevents writing or deleting data. The Anomaly Detector monitors the Audit Log for query volume spikes and unusual node-type access patterns; a compromised daemon issuing a full-graph dump will generate anomaly alerts unless the attacker also compromises the Audit Daemon, which is a separate process. The LUKS-encrypted graph image means the data is protected at rest the moment the daemon stops.

What this system cannot offer is runtime confidentiality against a compromised read daemon with an open database handle. No local architecture that provides queryable history can offer this guarantee: it is a structural property of the threat model, not a gap specific to this design. The correct response is layered mitigation and honest disclosure to the user that the Knowledge Graph, while protected against external breach, relies on the integrity of the Graph Daemon process itself.

Rate limiting and query complexity. Two attack vectors are addressed: denial of service via query flooding, and timing side-channels. Rate limiting is enforced per client identity using a token bucket: normal applications are limited to 100 queries per second with a burst limit of 200; the AI daemon has a higher sustained limit appropriate to its workload; system daemons are unlimited. Exceeding the rate limit generates an event for the Anomaly Detector, and repeated violations trigger an alert.

Query complexity limits prevent expensive traversals from being used as denial of service. Traversal depth is capped at 5 hops by default and never more than 10. Every query must include a `LIMIT` clause; queries without one are rejected before execution. Queries are aborted after 500 milliseconds; the client receives `QueryTimeout`. Response times have a small random noise value added to make statistical timing attacks on the graph impractical.

8.4.6 AI Security

The AI daemon has broader graph access than most applications by necessity, making it a higher-risk component with its own security model.

AI access levels. The user configures a global AI access level as the primary control over how much the AI can see. Table 18 lists the five levels. Action permissions, meaning what the AI can do rather than what it can read, are configured separately. Reading data and taking actions are fundamentally different risk categories and are therefore independently controlled. Suggest only mode means the AI proposes actions and the user manually executes them. Supervised mode means the AI executes actions but shows a preview with a cancellation window before proceeding. Autonomous mode means the AI can act within specific applications without per-action confirmation, and each application must be individually enabled for this; autonomous mode is never a global setting.

Table 18 AI read access levels

Level	Name	What the AI can access
0	Minimal	Nothing from the graph; only what the user explicitly pastes
1	Session-scoped	Only data from the current active session
2	Project-scoped	All data associated with a user-selected project
3	Time-scoped	Historical data up to a configurable lookback window
4	Full read	Everything the user has read access to

Prompt injection. Prompt injection is an attack where malicious instructions are embedded in content the AI processes with the intent of manipulating its behavior: a PDF might contain hidden text instructing the model to send files to an external server. Current language models cannot reliably distinguish between content they should process and instructions they should follow. This is a structural limitation, not a fixable bug. The defense is therefore architectural.

Every piece of content the AI receives is tagged with its origin before entering the prompt, so the model sees explicit markers distinguishing user input, external content such as PDFs, and graph data. Before external content reaches the main model, it passes through a small local model whose sole function is to detect AI-directed instructions; this is a probabilistic first pass that catches obvious injection attempts cheaply and locally.

As a hardcoded rule with no configuration surface, any action the AI wants to take that was triggered by external content requires explicit user confirmation regardless of the configured action permission level. An AI that read a malicious PDF cannot use that as a basis to execute commands without the user seeing exactly what it wants to do and why.

Documents are parsed in an isolated subprocess before their text content is passed to the AI. The subprocess has no network access and no access to the graph; only the extracted text leaves the sandbox, with no embedded scripts, no active content, and no metadata that could carry instructions.

These layers substantially reduce the attack surface and raise the cost of a successful injection, but they do not solve the underlying problem. Prompt injection is an open,

unsolved problem across the AI industry. No current system can guarantee that a sufficiently sophisticated injection embedded in external content will be detected or neutralized. This system's position is to minimize exposure, make AI-triggered actions visible and reversible, and be transparent about the remaining risk rather than to claim a solution that does not exist.

AI network proxy. The AI daemon has no direct network access. All cloud API calls go through a dedicated system proxy daemon that is the only component allowed to make outbound connections to AI provider endpoints. The AI daemon constructs a request, hands it to the proxy, and receives only the response. This prevents a compromised AI daemon from independently exfiltrating data over arbitrary network connections, ensures all AI-related network traffic is logged and attributable, and allows the proxy to enforce that only approved provider endpoints are contacted.

8.4.7 Audit Log

Every access to the Knowledge Graph, every AI action, and every permission grant or denial is recorded in the Audit Log. This serves two purposes: giving the user visibility into what is happening on their system, and providing the data needed for behavioral anomaly detection.

The Audit Log is an append-only ledger. Entries cannot be modified or deleted; only new entries can be added. Each entry is cryptographically linked to the previous one via a hash chain, so tampering with any historical entry is detectable by any reader that verifies the chain.

Two log tiers coexist. The Structural Log is always active and records interaction metadata without content: timestamp, application identity, which node types were queried, which relations traversed, result count, duration, and status. The query string itself, result contents, and concrete node IDs are never recorded in this tier. This is sufficient to detect suspicious behavior without being sufficient to reconstruct what data was accessed.

The Forensic Log is opt-in and time-limited. When activated in Settings, it additionally records the full query string, query parameters (not result contents), and the calling process stack trace. Forensic Mode is always limited to a maximum of 24 hours and then automatically deactivates. It is shown prominently as active in Settings while enabled and cannot be activated remotely by any application or administrator; only the user, locally.

A dedicated Audit Daemon is the sole writer to the log files. Other components send audit events over a restricted IPC channel and cannot write directly to the log. The Forensic Log is accessible exclusively to the user; no administrator, no daemon, and no application can read it. The Structural Log is readable by the user and by the Anomaly Detector for pattern analysis; in managed environments, administrators receive aggregated anomaly alerts only, never raw log contents.

Hash chain integrity and TPM anchoring. The hash chain makes retroactive tampering with individual log entries detectable by any reader that verifies the chain. It does not, however, prevent an attacker with early system access from deleting the log and

constructing a new chain from scratch over fabricated entries, because without an external anchor the chain-start value is self-referential.

To close this gap, the HMAC key used to sign Audit Log entries is derived from a secret sealed to the TPM against the measured boot state (TPM 2.0 PCR sealing). The TPM releases the key only when the system booted in the exact state recorded at seal time: the same bootloader, the same kernel, the same initramfs. An attacker who wants to forge a clean audit chain must reproduce the exact measured boot state that authorized the signing key, which requires the Secure Boot signing key. The TPM sealing described here uses the same PCR infrastructure already required for FDE key release in Section 8.4.2; no additional hardware is needed.

On systems without a TPM the HMAC key falls back to a software-derived secret stored in the Kernel Keyring. This preserves the chain-integrity guarantee against post-hoc entry modification but not against early-access forgery. The Security section of Settings indicates which protection level is active; this difference is disclosed to the user.

8.4.8 Anomaly Detection

The Anomaly Detector is a system daemon that reads the Audit Log and identifies unusual patterns. It is not a traditional antivirus and does not match file signatures or scan for known malware. It is a behavioral analysis tool that establishes a baseline of normal behavior on this specific machine and flags deviations.

Signature-based detection requires a continuously updated database of known threats and cannot detect novel attacks. Because this system already maintains a rich picture of normal behavior via the Knowledge Graph and Audit Log, behavioral detection is the natural fit: the data needed to define normal behavior already exists.

The Anomaly Detector watches for patterns including an application suddenly querying graph node types it has never queried before; query volume from an application spiking above its historical baseline; an AI action executed without a preceding user interaction in the same session; a network connection from a process that has never previously made network connections; a new eBPF program being loaded outside system update windows; and permission escalation requests at unusual times.

Alerts are surfaced via the notification system with enough context for the user to make a decision: which application, what it did, why it is unusual, and options to investigate or dismiss. The detector does not automatically block anything; it informs. Automatic blocking based on anomaly scores is an opt-in feature for managed environments where an administrator configures the policy. The Anomaly Detector runs locally and alert data does not leave the machine unless the user or administrator explicitly configures remote alert forwarding.

8.4.9 Network Security

On a conventional Linux system, any process can open a network connection to anywhere by default. This is the wrong default. On this system, outbound network access is denied unless explicitly permitted. Every application that needs network access declares it in its package manifest by protocol, destination, and direction;

connection attempts outside the declaration are blocked at the kernel level via eBPF-based network filtering and logged as security events.

VPN enforcement can be configured in managed environments such that certain network destinations are reachable only when a VPN is active. *WireGuard* [27] is the preferred VPN protocol. WireGuard is a modern VPN implemented directly in the Linux kernel (merged in 5.6), using Curve25519 for key exchange and ChaCha20-Poly1305 for encryption. Its design is deliberately minimal: the entire implementation is under 5,000 lines of kernel code, compared to tens of thousands for OpenVPN, making it easier to audit and maintain. Key management is simpler than IPsec: each peer has a static key pair, and peers are added by exchanging public keys. *OpenVPN* [64] support is provided for compatibility with existing enterprise infrastructure.

8.4.10 Physical Security

Full Disk Encryption handles the most common physical threat, but physical access threats go further.

A Cold Boot Attack exploits the fact that DRAM does not lose its contents instantly when power is removed. At low temperatures, an attacker who obtains a running or recently suspended device could potentially read the FDE key from memory. Two mitigations apply: the kernel is configured to overwrite RAM contents before suspend-to-disk, and AMD SME and Intel TME are hardware features available on modern CPUs that encrypt the entire RAM contents with a key that lives inside the CPU die and never leaves it. Even if an attacker physically removes the RAM modules, they read only ciphertext. Both are enabled by default where hardware supports them.

USBGuard [81] enforces a policy on USB device connections. When the screen is locked, no new USB devices are permitted to become active; only devices connected before the screen locked remain accessible. This blocks BadUSB attacks, where a specially crafted USB device presents itself as a keyboard and injects keystrokes. Enabled by default; specific devices such as hardware security keys can be whitelisted in Settings.

IOMMU [12] (Input-Output Memory Management Unit) restricts which memory regions each hardware device can access via Direct Memory Access (DMA). With IOMMU enabled, a malicious Thunderbolt or PCIe device can only access the memory regions the OS has explicitly mapped for it, not arbitrary system RAM. Enabled by default via kernel parameters. For Thunderbolt specifically, the default security level requires newly connected devices to be explicitly authorized by the user before they are activated; previously authorized devices are remembered.

TPM Remote Attestation allows a device to cryptographically prove to a remote server that it booted in a known-good state by sending a signed report of its measured boot PCR values. In a managed deployment this can gate network access: a device that fails attestation is denied VPN access until an administrator investigates. This is inspired by the *BeyondCorp* [87] model (Google's zero-trust network architecture, where no device or user is trusted by default based on network location alone) and represents a meaningful differentiator for enterprise deployments.

8.4.11 Memory Safety

Memory safety bugs are the most common source of exploitable vulnerabilities in system software and are structural problems with C and C++ that persist even with experienced developers. All system components developed for this project are written in Rust. The Rust compiler enforces memory safety at compile time through its ownership and borrow checker; an entire class of bugs that would be possible in C simply cannot compile in Rust. This is a compile-time guarantee, not a runtime check with overhead.

Third-party applications cannot be rewritten. Sandboxing limits what a compromised application can do; hardware-level mitigations provide a further layer. *Intel CET* [43] (Control-flow Enforcement Technology) adds a hardware-enforced shadow stack: a second, protected copy of return addresses that the CPU maintains separately from the regular call stack. On every function return, the CPU compares the return address from the regular stack against the shadow stack; a mismatch causes an immediate fault. This defeats the most common class of control-flow hijacking, return-oriented programming (ROP), where an attacker overwrites saved return addresses to chain together existing code fragments. CET is supported by the Linux kernel since version 6.6 and is enabled by default for new processes on supported hardware with no application changes required. For any C or C++ components compiled as part of this project, Clang's *CFI* [52] (Control Flow Integrity) instrumentation is applied. CFI adds compile-time instrumentation that validates at runtime that every indirect function call and virtual method dispatch targets only a function with the expected type signature; an attacker who corrupts a function pointer to point to an arbitrary location will cause an immediate abort rather than code execution.

8.4.12 Multi-User Isolation

When multiple users share a system, their isolation goes deeper than traditional Unix file permissions because the Knowledge Graph, AI configuration, and Audit Log introduce sensitive state beyond the home directory. Each user has a separate Kuzu instance, a separate Graph Daemon process running under their own system user, and a separate Audit Log. These are not shared resources with access control layered on top; they are separate instances that other users' processes cannot reach.

Home directories are managed via `systemd-homed`: each user's home directory is stored as a LUKS-encrypted portable image. When a user logs out, their home image is unmounted and the encryption key is discarded from memory. No other user, including root, can access the home image without the user's credentials. The Knowledge Graph lives inside the home directory and is therefore protected by this encryption as a structural consequence rather than an additional policy.

In a multi-user or managed environment, an administrator can install and remove system-wide software, configure network policies, set password and screen lock policies, view aggregated anomaly alerts, manage USBGuard whitelists, and manage Secure Boot keys. The administrator explicitly cannot read any user's Knowledge Graph, access any user's home directory contents, view any user's Audit Log, or intercept any user's AI sessions. These limitations are enforced by `systemd-homed` and

the per-user Graph Daemon architecture, not by policy alone. The administrator sees that something unusual happened via anomaly alerts; they do not see what.

8.5 Managed Environments

For enterprise deployments where this system runs on many machines administered centrally, additional infrastructure is required beyond what individual Signed Profile Packages provide. This section describes the managed environment architecture for organizations that need fleet-level control.

Policy Engine. A system daemon receives signed policy packages from a central management server and applies them locally. The daemon verifies the cryptographic signature of every package before applying any policy; unsigned or incorrectly signed packages are rejected outright. Policies can control permitted and blocked applications, network access rules beyond per-app declarations, AI provider restrictions including complete disabling of cloud AI, the USBGuard device whitelist, screen lock timeout and password complexity requirements, update scheduling so that updates are applied only during configured maintenance windows, and a minimum audit log retention period for compliance purposes.

LDAP and Active Directory. Most enterprises already have an identity management system, typically Microsoft Active Directory or an LDAP-compatible directory. Integration with these systems is a baseline requirement for enterprise adoption. *LDAP* (Lightweight Directory Access Protocol [70]) is the standard protocol for querying and modifying directory services such as Microsoft Active Directory and OpenLDAP. It stores organizational information: users, groups, computers, and their attributes. *Kerberos* [56] is the authentication protocol layered on top: it issues time-limited tickets that prove identity without sending passwords over the network; Active Directory uses Kerberos as its primary authentication mechanism.

User authentication, group membership, and basic policy delivery are handled via these standard protocols, which existing enterprise infrastructure already supports without proprietary agents on the management server side.

MDM-style management. Beyond LDAP, a modern MDM-style management layer provides richer capabilities: device enrollment, remote policy push, health status reporting, and remote wipe. This is a longer-term goal; LDAP compatibility is the foundation and MDM-style management is built on top of it once the base is stable.

Remote attestation as network gate. TPM Remote Attestation, described in Section 8.4.10, can be used as a network access gate in managed deployments. The workflow is:

1. The device boots and generates a TPM attestation report containing signed PCR values.
2. The device connects to the management server's attestation endpoint.

3. The server verifies the report against the expected baseline for that device's configuration.
4. If verification passes, the device receives its VPN credentials and network access.
5. If verification fails, the device is quarantined pending administrator review.

This ensures that a device with a modified bootloader, disabled Secure Boot, or an outdated security policy cannot silently access company resources. It is inspired by the BeyondCorp model [87] and is a meaningful differentiator for enterprise deployments.

GDPR and compliance tension. In managed environments there is an inherent conflict between user privacy rights and organizational compliance requirements. GDPR grants users rights over their personal data; many compliance frameworks in financial services, healthcare, and legal contexts require organizations to retain logs and audit trails. This system does not resolve the conflict, because it is a legal and organizational question rather than a technical one.

What the system does is make both sides technically possible. Users retain full visibility into and control over their own Knowledge Graph and personal Audit Log. Organizations can configure a minimum retention period for compliance-relevant audit events, which do not include Knowledge Graph contents. The separation between personal graph data, which is private to the user, and system audit events, which may be subject to organizational retention requirements, is maintained architecturally rather than by policy. Organizations deploying this system should seek legal counsel to define appropriate policies. The system provides the technical primitives; the legal framework is out of scope.

9 Companion App and Health

9.1 Companion App

The system ships with a first-party companion app for Android and iOS that bridges phone and desktop into a single coherent experience. It is built with Tauri 2.0 using the same Rust backend, the same design token system, and the same UI component library as the desktop. The companion app looks and feels like the OS because it is built from the same parts.

The protocol is custom. *KDE Connect* [45] is a widely used open-source protocol and application that links Linux desktops and Android phones: clipboard sync, notification mirroring, file transfer, and remote input. It was evaluated and rejected as the foundation for this project. Its protocol encodes assumptions appropriate for a bolt-on integration layer – such as per-feature capability negotiation at the session level rather than at pairing time – that would obstruct the deeper integration features planned for later tiers. Building on top of it would mean working around its assumptions rather than designing for the actual requirements.

Feature tiers. Features are organized into three trust tiers established at pairing time. Table 19 shows the breakdown. The distinction between paired and trusted exists because Tier 2 features involve access to graph context and health data, which require a higher trust level than file transfer and notification mirroring. Tier 2 is restricted to the official app; a third-party client that implements the pairing protocol only gets Tier 1.

Table 19 Companion app feature tiers

Tier	Trust level	Features
1	paired	File Drop; clipboard sync (manual trigger only, not automatic); notification mirroring; media playback control
2	trusted	Handoff (continue reading, browsing, or editing across devices); Unified Timeline (cross-device activity, local only); graph presence (phone sends active app and content context); health data forwarding
3	Planned	Wearable data pipeline; extended context sharing

Protocol and transport. Discovery uses *mDNS* [24] (Multicast DNS) service advertisement on a shared WLAN. mDNS is the same zero-configuration discovery protocol used by Apple's Bonjour and by Avahi on Linux; it allows devices to announce themselves by name on a local network segment without a central DNS server. Without a shared network, the desktop and phone locate each other via *BLE* (Bluetooth Low Energy [6]) presence broadcast carrying the device ID. BLE is a mode of Bluetooth designed for low-power periodic broadcasting; a device can transmit small advertisement packets continuously at minimal battery cost without maintaining a connection. Transport is selected by availability at connection time, shown in Table 20.

Table 20 Companion app transport selection

Transport	Use case
HTTPS over LAN	Primary transport; full feature set
WiFi Direct [89]	Large transfers without a shared router. WiFi Direct allows two devices to form a peer-to-peer 802.11 connection without a router or access point, at full WiFi throughput.
Bluetooth	Small data only: clipboard, notifications, control commands

Pairing uses a Trust on First Use model. Both devices generate an *Ed25519* [44] keypair on first launch. Ed25519 is an elliptic-curve digital signature scheme based on Curve25519; it produces 64-byte signatures, is fast to compute and verify, and has no known weaknesses from weak random number generation (a historical problem with earlier elliptic-curve schemes like ECDSA). The first connection uses *TLS* [68] (Transport Layer Security, the protocol underlying HTTPS) with self-signed certificates rather than certificates issued by a certificate authority. Self-signed certificates cannot

be validated by the standard CA chain, but here that is intentional: the validation happens out-of-band via a six-digit confirmation code displayed on the desktop and confirmed on the phone. This code is a hash of both devices' public keys; confirming it proves that the TLS session is between the two expected devices and not an attacker in the middle. After pairing, reconnection is automatic with no further confirmation. Unpair deletes the key on both sides with no trace.

Auto-connect. Paired devices broadcast BLE presence in the background. When a known device is detected, the system connects automatically using the best available transport. No user action is required after initial pairing. Android supports full background BLE advertising reliably. iOS imposes *CoreBluetooth Background Mode* [3] restrictions: background BLE scanning is not permitted continuously, only in periodic system-managed windows. Auto-connect works but is less reliable and carries a marginally higher battery cost on the iPhone side.

Theme synchronization. Because the companion app uses the same design token system as the desktop, the active theme propagates to the phone automatically: accent color, dark and light mode, typography. This is a structural side effect of the shared token infrastructure, not a feature that was separately implemented.

iOS limitations. iOS imposes constraints that cannot be worked around. There is no persistent background BLE scanning, which makes auto-connect less reliable and requires more manual reconnects than on Android. WiFi Direct is not available to third-party iOS apps. USB-C wired transport is inaccessible to third-party iOS apps without *MFi* (Made for iPhone/iPad/iPod) certification [5], Apple's hardware licensing program. Apps without MFi certification cannot use the Lightning or USB-C accessory protocol for direct data transfer. Health data forwarding via *HealthKit* [4] (Apple's on-device health data framework, accessible to third-party apps with explicit user consent) is feasible with explicit user permission.

Tier 1 works well on iOS. Tier 2 is functional but degraded. iOS is explicitly a secondary platform and this limitation is communicated honestly in the app and in documentation rather than obscured.

9.2 Health and Wellbeing

The system includes a first-party health data component: a local daemon that aggregates, stores, and optionally analyzes health and wellbeing data from connected sources. This is not a fitness application. It is health data infrastructure: an open, local equivalent of Apple Health or Google Fit, built as a system component with the same privacy guarantees as the rest of the platform.

Data architecture. Health data lives in a dedicated local store managed by a *Health Daemon*, separate from the Graph Daemon. Data never leaves the device without an explicit user action. Clinical data uses *FHIR R4* [42] (Fast Healthcare Interoperability Resources, the international standard for structured health data interchange

maintained by HL7). FHIR represents health records as typed REST resources – Observation, Condition, MedicationRequest, and others – with a defined JSON and XML serialization. Hospital and clinic systems across the EU are increasingly FHIR-capable, particularly following regulatory pressure in Germany (MIO/DiGA) and the broader eHealth interoperability mandates; using FHIR natively enables direct import and export without format conversion. Non-clinical data such as steps, sleep stages, heart rate variability, and nutrition uses an open JSON schema with documented field definitions. There is no proprietary format; full export is always available.

Data sources. The initial release supports manual input and phone sensors forwarded via the Companion App: steps, GPS, and screen time. The plugin-based architecture is designed from the start to accommodate further sources without structural changes: wearables via the Companion App, hospital and clinic FHIR import, and third-party applications via a documented Health API gated by the standard permission system.

Wearable pipeline. Wearables connect to the system through the Companion App rather than directly, using the wearable manufacturer's existing phone SDK on the phone side. The data path is:

```
Wearable -> Companion App (phone, manufacturer SDK)
         -> OS Health Daemon (via Companion protocol)
```

Listing 16 Wearable data path

No direct wearable protocol implementation is required in the OS. The Health Daemon's plugin interface is designed to accept forwarded wearable data from the start, so that adding wearable sources later does not require structural changes. This is planned architecture, not initial release scope.

Graph integration. Health data integration with the Knowledge Graph is strictly opt-in and split into three explicit levels, all of which default to off. Table 21 lists the levels. Each level requires the previous one; the Settings UI enforces this and does not allow incompatible states. The AI layer's health-adjacent features are gated entirely on these toggles. With all three off, the health module is a pure local viewer and aggregator with no AI involvement whatsoever.

Table 21 Health graph integration levels

Level	What it enables
1 - Health Nodes	Health events appear as standalone nodes in the Knowledge Graph
2 - Correlations	Graph links health data to activity data by time window (e.g. sleep quality correlated with focus session length)
3 - Pattern Recognition	Full cross-domain analysis across health, productivity, and communication data

Settings and permissions. Health has its own dedicated section in Settings rather than being buried under general privacy settings. Controls include per-source enable and disable for each data source independently; per-data-type enable and disable so the user can, for example, share step data with an app but not sleep data; the graph integration toggles from Table 21 presented as a grouped sub-section with clear explanations of what each level enables; a configurable data retention period per data type; full FHIR and JSON export; and full deletion with a confirmation step.

Third-party applications request health data access per data type through the standard permission system, but health-specific confirmation dialogs are more explicit about data sensitivity than generic permission prompts. A fitness app requesting sleep data gets a dialog that names the data type and explains clearly that this is health information, not a generic file access request.

10 Developer Experience, Sustainability, and Roadmap

10.1 Developer Experience and Infrastructure

The development infrastructure is designed around one constraint: a project of this scope spans many components with cross-cutting concerns, and the tooling must make that manageable without forcing artificial coupling between parts that should evolve independently.

Repository structure. The project uses a multi-repo layout under a single GitHub organization. Each component has its own repository, its own release cycle, and its own CI pipeline. Components can be updated independently as long as they do not introduce breaking changes to the interfaces they expose. A breaking interface change requires a coordinated update across all affected components, enforced by the versioning mechanism described below.

```
org/
  blueprint <- architecture documents and this document
  kernel-layer <- eBPF programs, Event Bus daemon
  knowledge <- Kuzu integration, graph schema, Graph Daemon
  ai-layer <- provider abstraction, MCP client
  compositor <- Smithay-based Wayland compositor (Rust)
  desktop-shell <- taskbar, launcher, shell (Tauri + TypeScript)
  ui-kit <- shared component library (TypeScript + Tailwind)
  os-sdk <- shared Rust crate for system integration
  app-files <- File Manager
  app-settings <- Settings
  app-terminal <- Terminal
  app-store <- Store
  module-sdk <- SDK for shell module development
  themes <- GTK4, Wine .msstyles, token source
  distro <- build system, ISO generation, meta-packages
```

Listing 17 Repository structure

Build system. *mkosi* [75], maintained by the systemd team, is the target tool for image generation. It builds bootable OS images from a declarative configuration file that specifies which packages to include, which overlay files to add, and how to configure the system. Unlike Kickstart or Kiwi, mkosi is designed around the systemd ecosystem: it integrates directly with systemd-repart for partition layout and produces well-formed UEFI images that support Secure Boot and Unified Kernel Images without custom glue scripts. It supports multiple output formats: ISO, raw disk image, and container. It integrates cleanly with the Open Build Service. Confirming that mkosi handles the full build pipeline in practice is an open item pending actual use.

The *Open Build Service* [59] (OBS) handles packaging for all custom components. OBS is an openSUSE-hosted build infrastructure that takes RPM spec files and produces binary packages for multiple distributions and architectures in parallel, without requiring the developer to provision build machines for each target. Packages are signed with the project GPG key – *GPG* (GNU Privacy Guard [40]) is the standard open-source implementation of the OpenPGP signing and encryption standard; package signatures let end-user systems verify that a package was built and signed by this project and has not been tampered with. OBS hosts the resulting package repository that end-user systems pull from. Both Fedora and OpenSUSE targets can be built from the same spec files, which is one practical reason both remain viable candidates.

Update system. Updates arrive from two independent streams managed separately. The base system, meaning the kernel and base distribution packages, is handled by the native package manager (dnf or zypper depending on the chosen base), with Snapper automatically taking a Btrfs snapshot before every update. Project components, meaning the compositor, shell, event bus, graph daemon, AI layer, and first-party applications, are distributed via the project's own OBS repository as RPM packages managed as a coordinated meta-package.

The update behavior is designed to be non-intrusive. Security updates for both streams are downloaded automatically in the background and applied at the next shutdown or reboot, never during an active session. No forced reboots, no countdown timers. All other updates are downloaded silently in the background; a single calm notification appears when they are ready. The user applies them when convenient. In managed environments, administrators can configure mandatory update windows; even then, no mid-session interruption occurs.

Rollback is available for every update: Btrfs and Snapper record a snapshot before each change, and the user can boot into a previous snapshot from the boot menu and roll back the entire system without any additional tooling.

Project components communicate over defined IPC interfaces. A breaking interface change increments the major version number and must be shipped as a coordinated meta-package update, so that incompatible component versions cannot coexist. Components verify interface compatibility at startup:

```
fn check_compatibility() -> Result<()> {
    let daemon_version = get_graph_daemon_version()?;
    if daemon_version.major != EXPECTED_MAJOR {
        bail!("Graph Daemon interface version mismatch. Run system update.");
```

```
    }
    Ok(())
}
```

Listing 18 Interface compatibility check at startup

A component that detects a version mismatch refuses to start and surfaces a clear message. The only resolution is applying the pending system update.

CI/CD pipeline. Tests run at three levels of integration. Table 22 shows when each level runs.

Table 22 CI trigger summary

Trigger	What runs
Every push or pull request	Unit tests for the affected repository
Pull request merge to main	Unit tests, lint, and build check
Daily at 02:00	Cross-component integration tests against all main branches
Release candidate	Full VM tests and smoke tests
kernel-layer push	Unit tests plus VM-based eBPF tests

Unit tests run per repository against mocks from os-sdk. Rust components use cargo test; TypeScript components use *Vitest* [85], a Vite-native unit test framework with Jest-compatible API and fast execution via native ES modules. No PR merges without green unit tests. The kernel-layer is a special case: eBPF logic that can be tested without a real kernel runs on the host as normal Rust unit tests, while eBPF programs themselves run in a minimal QEMU instance that CI spins up automatically.

Cross-component integration tests verify that two or more components communicate correctly over their real IPC interfaces. They run on the host without a full system boot, starting only the directly involved daemons as real processes against real IPC sockets. A failing integration test with all unit tests passing means interface drift between repositories, which is the primary failure mode this level exists to catch. Examples: the Event Bus emits a FileAccessEvent and the Graph Daemon writes correctly to Kuzu; a theme token change in Settings propagates to all running Tauri processes; the shell sends a keybinding action and the Input Manager calls the correct handler.

Full VM tests boot a complete system image in QEMU and run smoke tests: system boots without crash; Spotlight opens, finds an application, and launches it; eBPF events arrive and land in the Knowledge Graph; theme switch propagates correctly; module install and uninstall behave as expected. These tests are slow and run on release candidates only. The VM image is built by the same script used for real releases; no separate test image exists.

Local development workflow. Development happens at three levels of integration, each suited to a different class of work. The common case is developing a single component in isolation. Each repository has its own development setup with cargo

run, tauri dev, or hot-reload as appropriate. Components that use the mocks from os-sdk need no external dependencies running. Fast feedback loop, runs entirely on the developer's machine.

For shell and compositor work, Smithay supports nested mode: the compositor runs as a regular Wayland window inside the developer's existing desktop. The full shell appears inside that window. No VM or extra hardware is needed. This covers the majority of shell and compositor development; eBPF and kernel-level components are not active in nested mode.

Full system work requires *QEMU* [22], a machine emulator and virtualizer. QEMU can emulate a complete x86-64 machine – CPU, RAM, storage, network, and display – entirely in software, or accelerate execution via KVM (the Linux kernel's hardware virtualization support) to near-native speed when running on matching hardware. This covers eBPF development, the boot sequence, the update system, and anything that requires a real kernel or a complete system environment. The distro repository contains a script set for this workflow, including build-image.sh for a complete bootable image, run-vm.sh for an interactive session, run-vm.sh –share ~/projects/kernel-layer to mount a host directory into the VM via *virtiofs* [84] (a shared filesystem protocol that exposes a host directory to a QEMU guest at near-native speed using the virtio transport), so that the developer can compile on the host and test in the VM without a full image rebuild, and snapshot management scripts for saving and restoring VM state.

A strict rule applies to eBPF development: eBPF programs are developed and tested exclusively inside the VM, never directly on the development machine. A bad eBPF program can destabilize the kernel; the isolation of the VM is not optional.

Mock strategy. Every IPC interface in the system has an official mock implementation that lives in os-sdk directly alongside the interface definition. Mock and real implementation share the same type definitions. An outdated mock is a build error, not a runtime surprise: if an interface changes, the mock must be updated in the same pull request.

```
#[cfg(test)]
use os_sdk::graph_daemon::mock::GraphDaemonMock as GraphDaemon;

#[cfg(not(test))]
use os_sdk::graph_daemon::client::GraphDaemonClient as GraphDaemon;
```

Listing 19 Conditional mock usage in tests

There are no per-repository custom stubs and no fixture files that can drift from the real schema. The mock is the contract.

Project planning tooling. A project of this scope has cross-cutting concerns that span multiple repositories simultaneously: an Epic about the Spotlight Module API touches desktop-shell, os-sdk, module-sdk, and potentially compositor. Standard per-repository issue tracking is not sufficient for this. The requirements are hierarchical issues or Epics that span multiple repositories, dependency tracking between issues, a roadmap view across all components, and cross-repository milestone coordination.

Three options are under evaluation. The first is GitHub Projects v2 with a meta-repo convention: cross-cutting Epics live as issues in the blueprint repository and component repositories link back to the relevant Epic; GitHub Projects v2 aggregates issues from all repositories into one board. The advantage is zero additional tooling and everything staying in GitHub. The disadvantage is that dependency tracking is honor-system only with no enforcement, there is no proper roadmap view, and Epic-to-child linking is manual.

The second option is Linear, a proprietary hosted tool with strong multi-repo GitHub integration, real Epics with child issues, a dependency graph, and a roadmap view. It has the best user experience for exactly this use case but is proprietary, hosted externally, and costs money at team scale.

The third is Plane, an open-source Linear alternative that can be self-hosted. Feature parity with Linear is growing but not yet complete. Its advantage is that self-hosting fits the project's digital sovereignty philosophy with no external dependency; the disadvantage is additional operational overhead and some features that are still maturing.

The decision is not yet made. The requirement is clear: cross-repository Epic tracking with dependency enforcement and a roadmap view. GitHub Projects alone is likely insufficient for a project of this scope.

Documentation. Two documentation sites are maintained, both self-hosted. The user-facing wiki at wiki.[project].dev is written by hand in the spirit of the Arch Wiki: prose that explains intent and reasoning, not only what a setting does. It covers concepts, how-tos, troubleshooting, and architecture explanations. Community contributions are accepted via pull requests reviewed by the core team. The wiki source lives in the blueprint repository alongside the architecture documents.

The developer API reference at api.[project].dev is auto-generated from code comments: rustdoc for Rust components, typedoc for TypeScript components, aggregated and hosted under a single domain so there is one place to search across all components. Keeping it current is a CI gate: a pull request that removes or modifies a public API without updating its documentation comment does not merge.

10.2 Sustainability and Business Model

This is an open source project. The code is and remains open; that is a foundational commitment, not a marketing position. At the same time, building and maintaining a full operating system is a serious engineering effort that requires sustained funding. The goal is not to extract value from users but to find revenue streams that are structurally aligned with the project's values: user sovereignty, transparency, and European digital autonomy.

Principles. Four constraints rule out entire categories of business models upfront. Core OS functionality is not feature-gated behind a paywall; the desktop, the permission system, the Knowledge Graph, and the AI layer are open and available to everyone. There is no advertising, no telemetry sold to third parties, and no data brokering.

Venture capital with growth-at-all-costs expectations is incompatible with a project built on user trust and is therefore excluded. Enterprise features are an acceptable source of revenue; making the core worse for non-paying users is not.

Store revenue. The Store's tiered commission model, described in Section 7.5, generates revenue proportional to ecosystem health. The project earns meaningfully only if developers earn well first. At early stage this is negligible. At meaningful adoption it becomes a significant funding stream. The Store handles application sales, theme sales, module distribution, and profile package distribution under a single infrastructure.

Enterprise services. The Signed Profile Package system described in Section 8.2 is already designed with organizations in mind. The .lenv format is open and documented; anyone can author packages. Organizations deploying managed environments at scale will often want professional help: authoring and maintaining custom packages, deployment tooling, update management, and support SLAs. This is a natural B2B offering that does not touch the open source core. The analogy is Red Hat selling support for software that anyone can download for free.

OEM and hardware partnerships. At sufficient maturity and hardware compatibility, small European laptop manufacturers may want to ship the system pre-installed. The structural alignment is good: the manufacturer gets a differentiated product, users get a well-tested hardware and software combination, and the project gets sustainable revenue from certification and validation work and an ongoing support relationship. This requires a stable release and a polished out-of-box experience first and is a longer-term play. Relevant candidates in the European market include *Tuxedo Computers* [79], the *PINE64* [65] ecosystem, and potential *Framework* [32] partnerships.

Support tiers. A support subscription aimed at power users, small businesses, and developers who want guaranteed response times and direct access offers prioritized bug reports, LTS release guarantees with defined support windows, and a direct communication channel with the core team. No features are locked behind this; it is a service layer on top of the open product, priced modestly enough to be accessible to individual developers.

Public funding and grants. The European open source and digital sovereignty funding landscape is underutilized by most projects. The *Sovereign Tech Fund* [73] in Germany funds open source infrastructure projects and explicitly values work that reduces dependency on US technology monopolies. The *NLnet Foundation* [57] funds internet and open source projects with a privacy and user rights focus and has supported numerous small to medium projects. The *NGI* (Next Generation Internet [29]) programme runs multiple EU funding calls per year for open source, privacy, and decentralization work.

The European digital autonomy angle is not only a philosophical position; it is a genuine funding argument. A European open source operating system built on sovereignty principles is precisely what these programmes exist to support. Grants do

not provide recurring revenue but they can fund specific development phases without creating investor obligations or governance compromises.

Path to incorporation. A company makes sense when there is external money to manage, not before. Premature incorporation adds administrative overhead with no corresponding benefit. A realistic sequence: the project gains traction and a community forms; grant applications are filed; Store revenue starts, providing the first external money flow; the first enterprise inquiry for deployment support arrives as the first B2B signal. At that point incorporation becomes appropriate, probably as a GmbH in Austria or Germany, or as a European cooperative structure if community governance is a priority.

The company's role is to employ core contributors, manage the Store infrastructure, and sell enterprise and support services. The open source project remains governed independently of the company: contribution policy, release decisions, and core architecture are not dictated by commercial interests. This separation must be written down explicitly before the company is incorporated, not negotiated afterward under time pressure.

10.3 Roadmap

The roadmap is intentionally sparse at this stage. Defining detailed phase scope before the foundational components are proven would produce a plan that reflects assumptions rather than reality. Phases 1 through the public release will be scoped as Phase 0 concludes and the first components are built and measured.

Phase 0 - Foundation (current). Architecture and design decisions are documented in this blueprint. The proof of concept consists of the Event Bus and Knowledge Graph running as a daemon and collecting basic system events. The base distribution decision between Fedora and OpenSUSE Slowroll is pending and will be made before Phase 1 begins. The Kuzu benchmark described in Section 4.5 at 5,000 events per second is a Phase 0 exit criterion.

Phase 1 and beyond. Phase scope will be defined once Phase 0 is complete. The ordering of subsequent phases follows the architectural dependency graph: kernel layer and event infrastructure before the graph, the graph before the AI layer, the compositor before the shell, the shell before Store and module infrastructure. Public release comes after a stable, tested combination of all layers exists.

References

[1] Anthropic. *Anthropic API Reference*. URL: https://docs.anthropic.com/en/api/getting-started.

[2] Anthropic. *Model Context Protocol Specification*. 2024. URL: https://spec.modelcontextprotocol.io.

[3] Apple Inc. *Core Bluetooth Background Execution Modes*. URL: https://developer.apple.com/documentation/corebluetooth/about-core-bluetooth.

[4] Apple Inc. *HealthKit*. URL: https://developer.apple.com/documentation/healthkit.

[5] Apple Inc. *MFi Program*. URL: https://developer.apple.com/programs/mfi/.

[6] Arch Linux Wiki contributors. *Bluetooth*. Arch Linux Wiki. URL: https://wiki.archlinux.org/title/Bluetooth.

[7] Arch Linux Wiki contributors. *Btrfs*. Arch Linux Wiki. URL: https://wiki.archlinux.org/title/Btrfs.

[8] Arch Linux Wiki contributors. *Direct Rendering Manager*. Arch Linux Wiki. URL: https://wiki.archlinux.org/title/Direct_Rendering_Manager.

[9] Arch Linux Wiki contributors. *dm-crypt*. Arch Linux Wiki. URL: https://wiki.archlinux.org/title/Dm-crypt.

[10] Arch Linux Wiki contributors. *Flatpak*. Arch Linux Wiki. URL: https://wiki.archlinux.org/title/Flatpak.

[11] Arch Linux Wiki contributors. *Kernel keyring*. Arch Linux Wiki. URL: https://wiki.archlinux.org/title/Kernel_keyring.

[12] Arch Linux Wiki contributors. *PCI passthrough via OVMF*. Arch Linux Wiki. URL: https://wiki.archlinux.org/title/PCI_passthrough_via_OVMF.

[13] Arch Linux Wiki contributors. *Snapper*. Arch Linux Wiki. URL: https://wiki.archlinux.org/title/Snapper.

[14] Arch Linux Wiki contributors. *systemd*. Arch Linux Wiki. URL: https://wiki.archlinux.org/title/Systemd.

[15] Arch Linux Wiki contributors. *systemd-homed*. Arch Linux Wiki. URL: https://wiki.archlinux.org/title/Systemd-homed.

[16] Arch Linux Wiki contributors. *Trusted Platform Module*. Arch Linux Wiki. URL: https://wiki.archlinux.org/title/Trusted_Platform_Module.

[17] Arch Linux Wiki contributors. *Unified Extensible Firmware Interface/Secure Boot*. Arch Linux Wiki. URL: https://wiki.archlinux.org/title/Unified_Extensible_Firmware_Interface/Secure_Boot.

[18] Arch Linux Wiki contributors. *Unified kernel image*. Arch Linux Wiki. URL: https://wiki.archlinux.org/title/Unified_kernel_image.

[19] Arch Linux Wiki contributors. *Wine*. Arch Linux Wiki. URL: https://wiki.archlinux.org/title/Wine.

[20] Arch Linux Wiki contributors. *XWayland*. Arch Linux Wiki. URL: https://wiki.archlinux.org/title/XWayland.

[21] aya-rs contributors. *aya: eBPF library for Rust*. URL: https://aya-rs.dev.

[22] Fabrice Bellard et al. *QEMU: A Generic and Open Source Machine Emulator and Virtualizer*. URL: https://www.qemu.org.

[23] Bundesamt für Sicherheit in der Informationstechnik. *SiSyPHuS Win10: Studie zu Systemdesign, Protokollierung, Härtung und Sicherheitsfunktionen in Windows 10*. Technical report. BSI, 2022. URL: https://www.bsi.bund.de/sisyphus.

[24] Stuart Cheshire and Marc Krochmal. *Multicast DNS (RFC 6762)*. 2013. URL: https://datatracker.ietf.org/doc/html/rfc6762.

[25] Brad Davis, Brad Peabody, and P. Leach. *RFC 9562: Universally Unique IDentifiers (UUIDs)*. 2024. URL: https://www.rfc-editor.org/rfc/rfc9562.

[26] Philip Doitsujin et al. *DXVK: A Vulkan-based translation layer for Direct3D*. URL: https://github.com/doitsujin/dxvk.

[27] Jason A. Donenfeld. *WireGuard: Fast, Modern, Secure VPN Tunnel*. URL: https://www.wireguard.com.

[28] eBPF Foundation. *What is eBPF?* URL: https://ebpf.io/what-is-ebpf/.

[29] European Commission. *Next Generation Internet (NGI)*. URL: https://ngi.eu.

[30] Fedora Project. *Fedora Linux*. URL: https://fedoraproject.org.

[31] Feral Interactive. *GameMode*. URL: https://github.com/FeralInteractive/gamemode.

[32] Framework Computer Inc. *Framework Computer*. URL: https://frame.work.

[33] freedesktop.org. *Desktop Notifications Specification*. URL: https://specifications.freedesktop.org/notification-spec/latest/.

[34] freedesktop.org. *XDG Base Directory Specification*. URL: https://specifications.freedesktop.org/basedir-spec/latest/.

[35] freedesktop.org. *XDG Desktop Portal*. URL: https://flatpak.github.io/xdg-desktop-portal/.

[36] Georgi Gerganov et al. *llama.cpp*. URL: https://github.com/ggml-org/llama.cpp.

[37] GNOME Project. *GNOME Online Accounts*. URL: https://wiki.gnome.org/Projects/GnomeOnlineAccounts.

[38] GNOME Project. *GTK: The GIMP Toolkit*. URL: https://gtk.org.

[39] GNOME Project. *libnotify*. URL: https://gitlab.gnome.org/GNOME/libnotify.

[40] GnuPG Project. *GNU Privacy Guard (GnuPG)*. URL: https://gnupg.org.

[41] Dick Hardt. *The OAuth 2.0 Authorization Framework (RFC 6749)*. 2012. URL: https://datatracker.ietf.org/doc/html/rfc6749.

[42] HL7 International. *FHIR R4: Fast Healthcare Interoperability Resources, Release 4*. 2019. URL: https://hl7.org/fhir/R4/.

[43] Intel Corporation. *Control-flow Enforcement Technology (CET) Preview*. URL: https://www.intel.com/content/www/us/en/developer/articles/technical/technical-look-control-flow-enforcement-technology.html.

[44] Simon Josefsson and Ilari Liusvaara. *Edwards-Curve Digital Signature Algorithm (EdDSA) (RFC 8032)*. 2017. URL: https://datatracker.ietf.org/doc/html/rfc8032.

[45] KDE Community. *KDE Connect*. URL: https://kdeconnect.kde.org.

[46] Khronos Group. *Vulkan Overview*. URL: https://www.vulkan.org.

[47] Konrad Kollnig et al. "Privacy of Default Apps in Apple's Mobile Ecosystem". In: *Proceedings of the CHI Conference on Human Factors in Computing Systems*. 2024. DOI: 10.1145/3613904.3642831.

[48] Kùzu Inc. *Kùzu: Embeddable Graph Database*. URL: https://kuzudb.com.

[49] Kenny Levinsen. *greetd*. URL: https://git.sr.ht/~kennylevinsen/greetd.

[50] libbpf contributors. *libbpf*. URL: https://github.com/libbpf/libbpf.

[51] libinput contributors. *libinput*. URL: https://wayland.freedesktop.org/libinput/doc/latest/.

[52] LLVM Project. *Clang Control Flow Integrity*. URL: https://clang.llvm.org/docs/ControlFlowIntegrity.html.

[53] MangoHud contributors. *MangoHud*. URL: https://github.com/flightlessmango/MangoHud.

[54] Mesa3D contributors. *The Mesa 3D Graphics Library*. URL: https://www.mesa3d.org.

[55] Microsoft Security Response Center. *A Proactive Approach to More Secure Code*. 2019. URL: https://msrc.microsoft.com/blog/2019/07/a-proactive-approach-to-more-secure-code/.

[56] Clifford Neuman et al. *The Kerberos Network Authentication Service (V5) (RFC 4120)*. 2005. URL: https://datatracker.ietf.org/doc/html/rfc4120.

[57] NLnet Foundation. *NLnet Foundation*. URL: https://nlnet.nl.

[58] Ollama contributors. *Ollama*. URL: https://ollama.com.

[59] Open Build Service contributors. *Open Build Service*. URL: https://openbuildservice.org.

[60] OpenAI. *OpenAI API Reference*. URL: https://platform.openai.com/docs/api-reference.

[61] openCypher contributors. *The Cypher Query Language*. URL: https://opencypher.org.

[62] openSUSE Project. *openSUSE Linux*. URL: https://www.opensuse.org.

[63] openSUSE Project. *SDB:Zypper usage*. URL: https://en.opensuse.org/SDB:Zypper_usage.

[64] OpenVPN Inc. *OpenVPN*. URL: https://openvpn.net.

[65] PINE64. *PINE64*. URL: https://www.pine64.org.

[66] Tom Preston-Werner. *TOML: Tom's Obvious Minimal Language*. URL: https://toml.io.

[67] Erik Reider. *SwayNotificationCenter*. URL: https://github.com/ErikReider/SwayNotificationCenter.

[68] Eric Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3 (RFC 8446)*. 2018. URL: https://datatracker.ietf.org/doc/html/rfc8446.

[69] RPM Software Management. *DNF, the next-generation replacement for YUM*. URL: https://dnf.readthedocs.io.

[70] Jim Sermersheim. *Lightweight Directory Access Protocol (LDAP): The Protocol (RFC 4511)*. 2006. URL: https://datatracker.ietf.org/doc/html/rfc4511.

[71] Shadcn. *shadcn/ui*. URL: https://ui.shadcn.com.

[72] Smithay contributors. *Smithay: A Wayland compositor library in Rust*. URL: https://smithay.github.io.

[73] Sovereign Tech Fund. *Sovereign Tech Fund*. URL: https://www.sovereigntechfund.de.

[74] System76. *cosmic-comp: The COSMIC Wayland compositor*. URL: https://github.com/pop-os/cosmic-comp.

[75] systemd contributors. *mkosi — Build Bespoke OS Images*. URL: https://github.com/systemd/mkosi.

[76] Tailwind Labs. *Tailwind CSS*. URL: https://tailwindcss.com.

[77] Tauri contributors. *Tauri*. URL: https://tauri.app.

[78] The Qt Company. *Qt Framework*. URL: https://www.qt.io.

[79] TUXEDO Computers GmbH. *TUXEDO Computers*. URL: https://www.tuxedocomputers.com.

[80] UEFI Forum. *UEFI Specification*. URL: https://uefi.org/specifications.

[81] USBGuard contributors. *USBGuard*. URL: https://usbguard.github.io.

[82] Valve Corporation. *Proton*. URL: https://github.com/ValveSoftware/Proton.

[83] Valve Corporation. *vkd3d-proton*. URL: https://github.com/HansKristian-Work/vkd3d-proton.

[84] virtio-fs contributors. *virtiofs: A Shared File System for Virtual Machines*. URL: https://virtio-fs.gitlab.io.

[85] Vitest contributors. *Vitest*. URL: https://vitest.dev.

[86] W3C Web Accessibility Initiative. *Web Content Accessibility Guidelines (WCAG) 2.2*. 2023. URL: https://www.w3.org/TR/WCAG22/.

[87] Rory Ward and Betsy Beyer. *BeyondCorp: A New Approach to Enterprise Security*. 2014. URL: https://research.google/pubs/beyondcorp-a-new-approach-to-enterprise-security/.

[88] Wayland protocols. *ext-session-lock-v1*. URL: https://wayland.app/protocols/ext-session-lock-v1.

[89] Wi-Fi Alliance. *Wi-Fi Direct*. URL: https://www.wi-fi.org/discover-wi-fi/wi-fi-direct.

[90] wlroots contributors. *wlr-protocols*. URL: https://gitlab.freedesktop.org/wlroots/wlr-protocols.

[91] WorkOS. *Radix UI*. URL: https://www.radix-ui.com.